

생성형 AI 활용

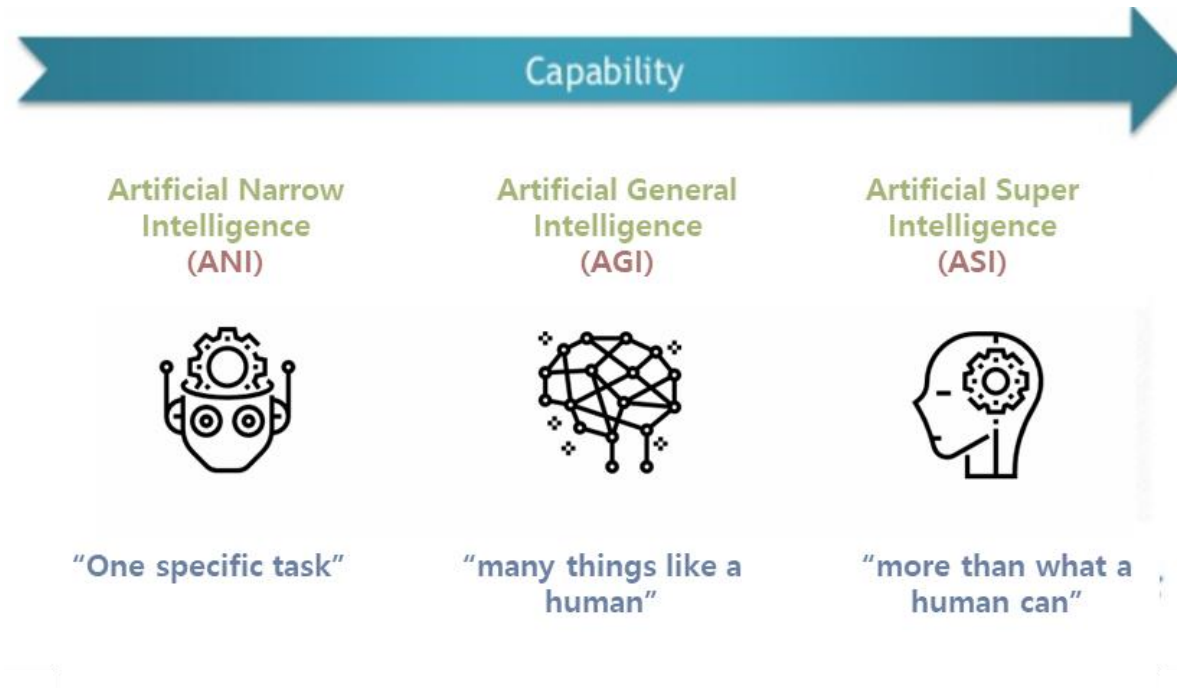
Dept. of Data Science, Kangwon National Univ.
Lee, Won Sang, *Ph.D*
Associate Professor
Director, Gangneung Startup Support Center
CEO, Unique AI

Table of Contents

- I. AI와 LLM 이해하기
- II. LLM 활용과 프롬프팅
- III. Work, Rag, Finetuning
- IV. Vibe coding 소개
- V. MCP 소개
- VI. LLM 실습

I. AI와 LLM 이해하기

- AI 개요

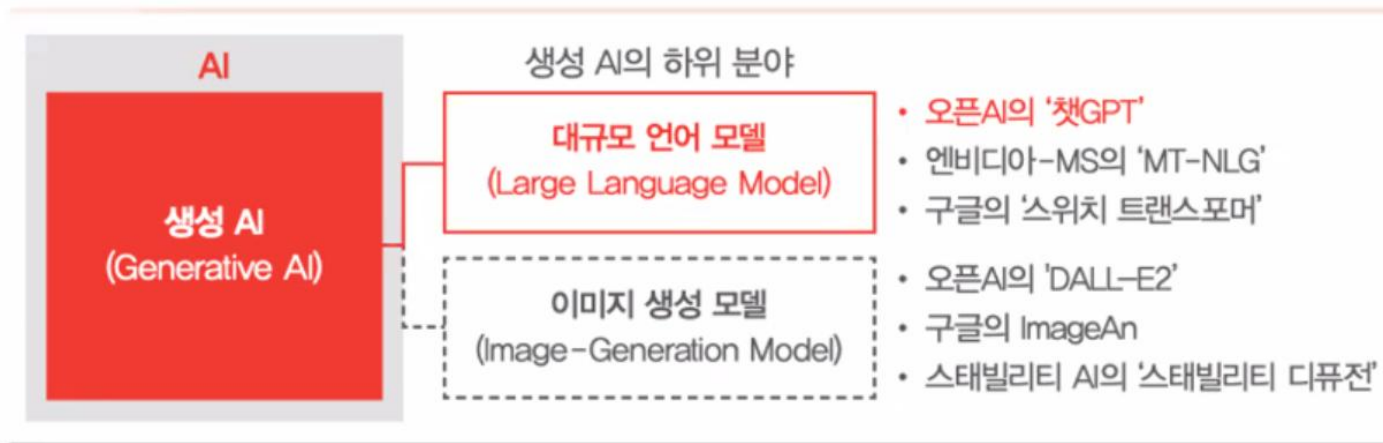


- '약인공지능(ANI)', '범용인공지능(AGI)', '초인공지능(ASI)'
- ANI: 주어진 데이터 내에서 주어진 문제를 해결하는 AI
- AGI로의 진입 가속화 중

I. AI와 LLM 이해하기

• 생성형 AI

- 대형 언어모델이나 이미지 생성모델 활용, User가 원하는 것을 생성하는 AI 분야
- LLM: 테라바이트 단위의 대용량 코퍼스로 모형을 학습, 문장을 생성하는 모형



※ 자료: 삼일PwC경영연구원

I. AI와 LLM 이해하기

- Scaling Laws (Open AI, 2020)
 - 컴퓨팅 리소스, 데이터, 모형 크기를 늘릴 수록 성능 개선
 - 새로운 능력이 생겨남 (Emergent Abilities)

Scaling Laws for Neural Language Models			
Jared Kaplan *		Sam McCandlish *	
Johns Hopkins University, OpenAI		OpenAI	
jaredk@jhu.edu		sam@openai.com	
Tom Henighan	Tom B. Brown	Benjamin Chess	Rewon Child
OpenAI	OpenAI	OpenAI	OpenAI
henighan@openai.com	tom@openai.com	bchess@openai.com	rewon@openai.com
Scott Gray	Alec Radford	Jeffrey Wu	Dario Amodei
OpenAI	OpenAI	OpenAI	OpenAI
scott@openai.com	alec@openai.com	jeffwu@openai.com	damodei@openai.com

Abstract

We study empirical scaling laws for language model performance on the cross-entropy loss. The loss scales as a power-law with model size, dataset size, and the amount of compute used for training, with some trends spanning more than seven orders of magnitude. Other architectural details such as network width or depth have minimal effects within a wide range. Simple equations govern the dependence of overfitting on model/dataset size and the dependence of training speed on model size. These relationships allow us to determine the optimal allocation of a fixed compute budget. Larger models are significantly more sample-efficient, such that optimally compute-efficient training involves training very large models on a relatively modest amount of data and stopping significantly before convergence.

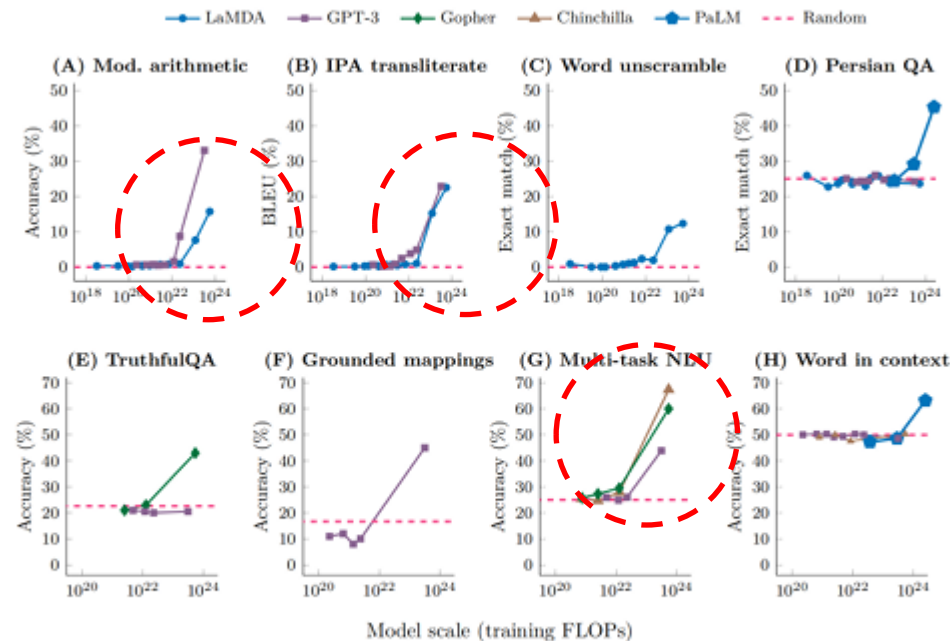
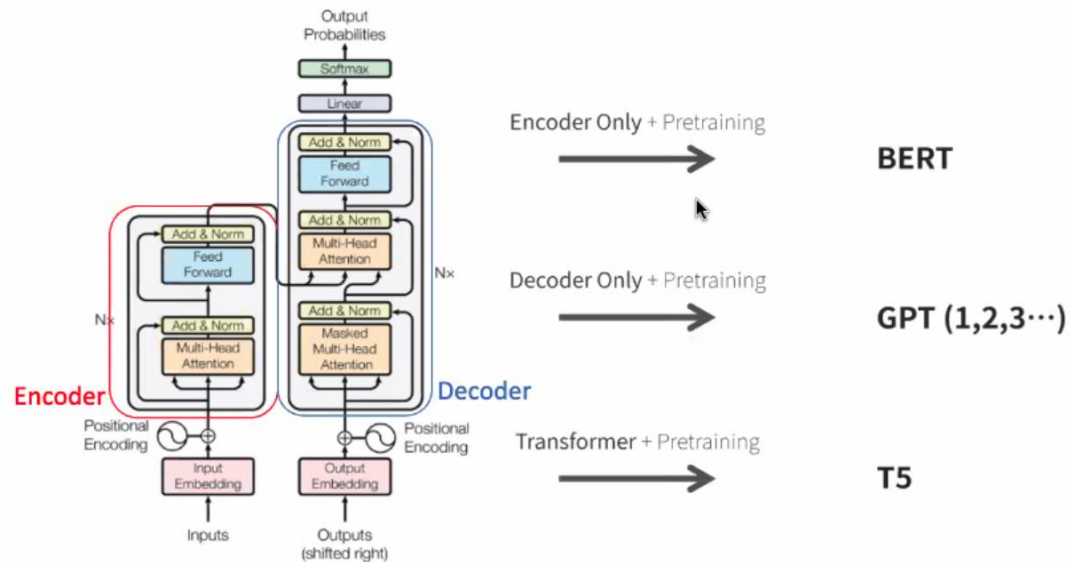


Figure 1: Emergent abilities of large language models. Model families display *sharp* and *unpredictable* increases in performance at specific tasks as scale increases. Source: Fig. 2 from [33].

I. AI와 LLM 이해하기

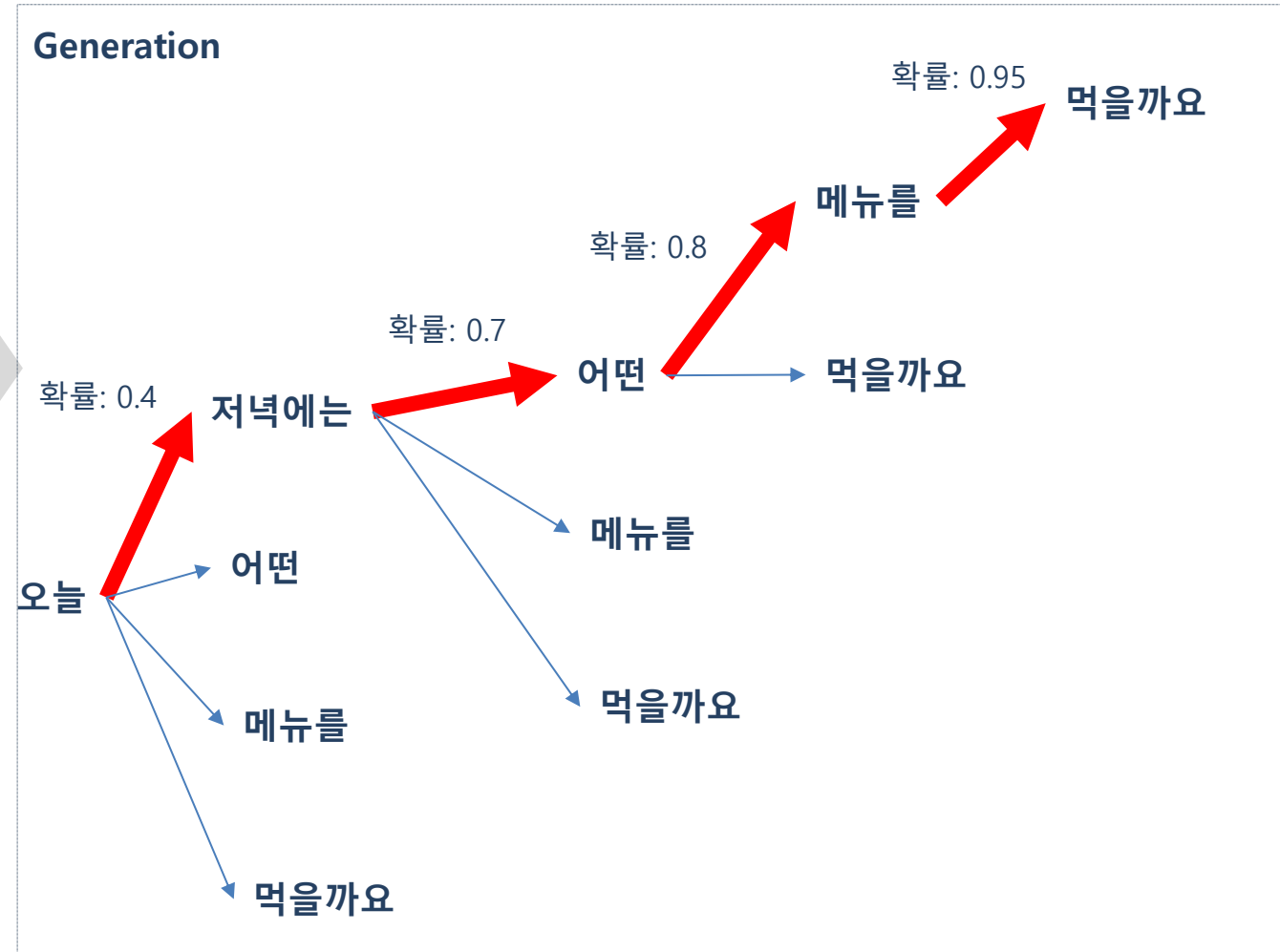
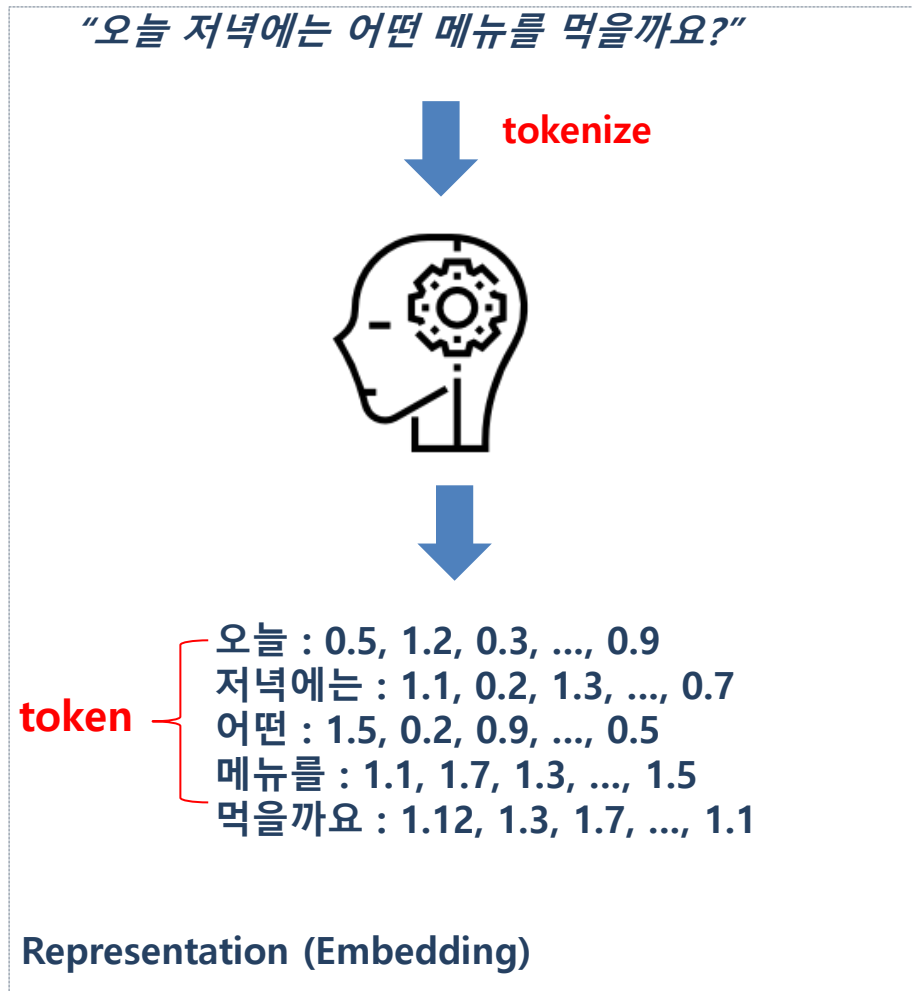
- 간단하게 이해하는 LLM

- 언어모형: 주어진 문장을 구성하는 단어에 대한 확률을 모델링
- 주어진 문장/단어의 다음 단어를 예측, 문장 내 Masking한 단어를 예측
- 대용량 텍스트를 학습한 모형이 좋은 Representation(Embedding)을 보유



I. AI와 LLM 이해하기

- 간단하게 이해하는 LLM



I. AI와 LLM 이해하기

- 다양한
- LLM

The screenshot displays the Hugging Face homepage, which is a central hub for AI models. The interface is divided into several sections:

- Header:** Features the Hugging Face logo, a search bar for models, datasets, and users, and navigation links for Models, Datasets, Spaces, Posts, Docs, Solutions, Pricing, Log In, and Sign Up.
- Left Sidebar (Tasks):** A vertical menu with tabs for Tasks, Libraries, Datasets, Languages, Licenses, and Other. Under the 'Tasks' tab, models are categorized into:
 - Multimodal:** Image-Text-to-Text, Visual Question Answering, Document Question Answering.
 - Computer Vision:** Depth Estimation, Image Classification, Object Detection, Image Segmentation, Text-to-Image, Image-to-Text, Image-to-Image, Image-to-Video, Unconditional Image Generation, Video Classification, Text-to-Video, Zero-Shot Image Classification, Mask Generation, Zero-Shot Object Detection, Text-to-3D, Image-to-3D, Image Feature Extraction.
 - Natural Language Processing:** Text Classification, Token Classification, Table Question Answering, Question Answering, Zero-Shot Classification, Translation, Summarization, Feature Extraction, Text Generation, Text2Text Generation, Fill-Mask, Sentence Similarity.
 - Audio:** Text-to-Speech, Text-to-Audio, Automatic Speech Recognition, Audio-to-Audio.
- Main Content Area (Models):** Displays a grid of model cards. Each card includes the model name, its primary task, and statistics such as the number of likes and updates. Examples of models shown include:
 - mistral-community/Mixtral-8x22B-v0.1**: Text Generation, updated about 10 hours ago, 33.9k likes, 595 hearts.
 - openbmb/MiniCPM-V-2**: Visual Question Answering, updated 2 days ago, 2.45k likes, 287 hearts.
 - HuggingFaceH4/zephyr-orpo-141b-A35b-v0.1**: Text Generation, updated 5 days ago, 3.91k likes, 194 hearts.
 - parler-tts/parler_tts_mini_v0.1**: Text-to-Speech, updated 7 days ago, 14.4k likes, 190 hearts.
 - HuggingFaceM4/idefics2-8b**: Image-Text-to-Text, updated about 5 hours ago, 5.23k likes, 173 hearts.
 - Vezora/Mistral-22B-v0.1**: Text Generation, updated 6 days ago, 519 likes, 139 hearts.
 - taide/TAIDE-LX-7B-Chat**: Text Generation, updated 1 day ago, 1.2k likes, 83 hearts.
 - mistralai/Mixtral-8x22B-v0.1**: Text Generation, updated about 11 hours ago, 5 likes, 69 hearts.
 - google/codegemma-7b-it**: Text Generation, updated 1 day ago, 5.29k likes, 131 hearts.
 - meta-llama/Llama-2-7b-chat-hf**: Text Generation, updated about 16 hours ago, 1.47M likes, 3.38k hearts.
 - CohereForAI/c4ai-command-r-plus**: Text Generation, updated 8 days ago, 153k likes, 1.15k hearts.
 - mistralai/Mixtral-8x22B-Instruct-v0.1**: Text Generation, updated about 8 hours ago, 124 likes, 202 hearts.
 - NexaAIDev/Octopus-v2**: Text Generation, updated 5 days ago, 10.7k likes, 728 hearts.
 - alpindale/WizardLM-2-8x22B**: Text Generation, updated 1 day ago, 2.04k likes, 175 hearts.
 - mistralai/Mistral-7B-Instruct-v0.2**: Text Generation, updated 25 days ago, 2.98M likes, 1.93k hearts.
 - Vezora/Mistral-22B-v0.2**: Text Generation, updated 3 days ago, 995 likes, 91 hearts.
 - amazingvince/Not-WizardLM-2-7B**: Text Generation, updated 1 day ago, 631 likes, 72 hearts.
 - mistralai/Mixtral-8x7B-Instruct-v0.1**: Text Generation, updated Feb 29, 846k likes, 3.67k hearts.
 - openai/whisper-large-v3**: Automatic Speech Recognition, updated Feb 8, 1.82M likes, 2.27k hearts.
 - runwayml/stable-diffusion-v1-5**: Text-to-Image, updated Aug 24, 2023, 4.07M likes, 10.7k hearts.

- **챗GPT**

- GPT3는 GPT2에 비해 10배 이상의 데이터(570GB), 100배 이상 모델 크기(175B)
- GPT4는 1700B의 파라미터(estimated)
- GPT5?
- Few-shot 학습을 통한 In-Context Learning 향상
 - 프롬프트의 내용만으로 Task 수행
 - 프롬프트 내 맥락을 이해하고 답변

II. LLM 활용과 프롬프팅

- 프롬프팅을 통한 업무활용: 이미지에서 텍스트 추출, 데이터 분석, 코딩 등...



Rowan Cheung
@rowancheung

The ChatGPT Mac app is the ultimate screenshot-to-code tool.

Screenshot anything, paste it in the ChatGPT shortcut, and just tell GPT-4o to code it for you.

Here's me taking a snapshot of Snake Game and getting fully working code in 90 seconds. Video is on 3x speed.

[Traducir post](#)



Zain Kahn
@heykahn

/ild. GPT-4o on ChatGPT can generate full blown charts and statistical analysis from spreadsheets with a single prompt in less than 30 seconds.

his stuff use to take ages in Excel.

[aducir post](#)

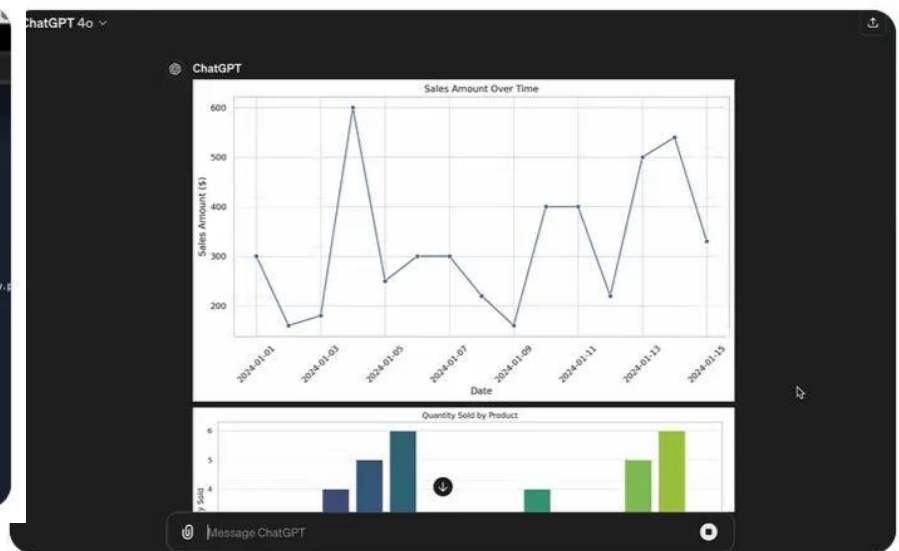
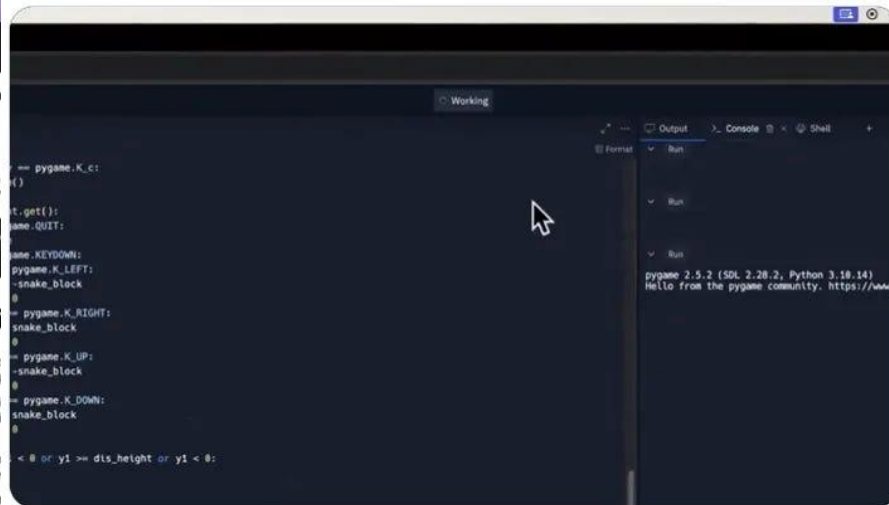
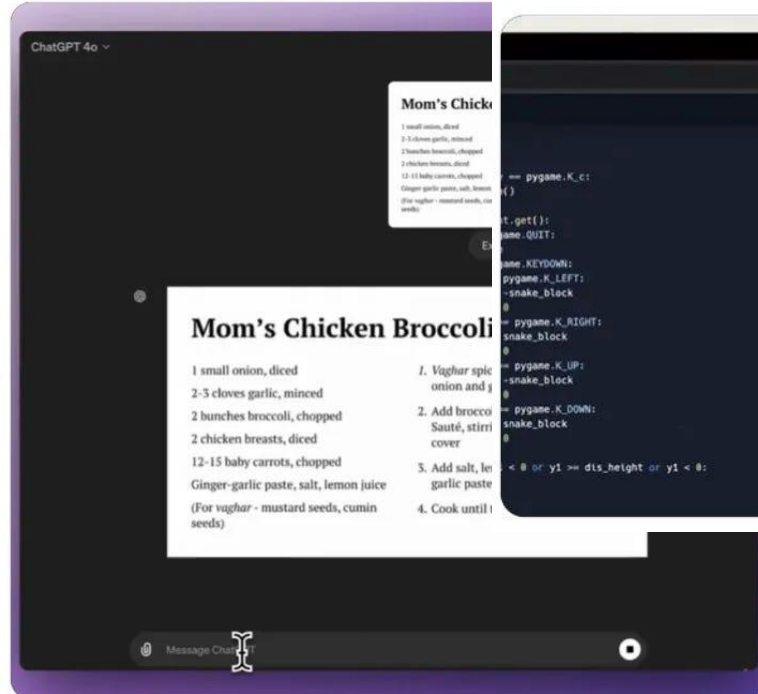


Robin Delta
@heyrobinai

3. Extract text from images

Prompt: Extract this recipe to a JSON file

[Traducir post](#)



II. LLM 활용과 프롬프팅

- **프롬프트 구조화**

- Delimiter와 프롬프트 내 영역 지정

- Delimiter: 특정 토큰으로 프롬프트에서 영역을 지정할 때 활용
 - ###, >>>, ===, <태그> 등으로 표현, 각 영역이 의도에 맞게 챗GPT에 전달되도록 함

- **활용 예시 프롬프트**

대화 예시와 결과 예시를 참고해서, 다음 <<<대화>>>에 나오는 각 문장의 감성을 '긍정' 이나 '부정' 으로 분류하시오.

###

대화 예시

[행인1]: 오늘 날씨가 좋네요

[행인2]: 좋긴 뭐가 좋아요

###

결과 예시

긍정

부정

###

<<<

[행인1] 비가 오니 차분합니다.

[행인2] 비오는 날 분위기 좋아요

>>>

II. LLM 활용과 프롬프팅

- 프롬프트 구조화
 - XML 태그 스타일의 영역 지정
 - XML태그?
 - XML은 데이터의 구조를 기술하는 Mark up 언어
 - 데이터의 각 항목에 대해 의미하는 태그로 마크업
 - 예: <키워드> 챗, 생성AI </키워드>

- 프롬프트 구조화

- 활용 예시 프롬프트(XML스타일)

대화 예시와 결과 예시를 참고해서, 다음 <<<대화>>>에 나오는 각 문장의 감성을 '긍정' 이나 '부정' 으로 분류하시오.

<범주>

긍정

부정

</범주>

<대화예시>

[행인1]: 오늘 날씨가 좋네요

[행인2]: 좋긴 뭐가 좋아요

</대화예시>

<결과예시>

긍정

부정

</결과예시>

<대화>

[행인1] 비가 오니 짜증납니다.

[행인2] 비오는 날 분위기 좋아요

</대화>

II. LLM 활용과 프롬프팅

- 프롬프트 구조화

- 시스템 프롬프트 = 맞춤형 지침

- “답변의 규칙 ”

- 챗GPT가 어떻게 응답해야 하는지를 알려주는 지시사항 또는 가이드에 대한 프롬프트
 - 입력하려는 질문 앞에서 제시함
 - 이후 나오는 프롬프트에 대해 지속적으로 적용

- 주요 내용

- 태스크 정의
 - 출력 형식 정의
 - 준수사항 정의: 피해야 할 응답 등

II. LLM 활용과 프롬프팅

- 프롬프팅 관련 기능: 메모리, 아카이브, 컨텍스트, 챗
 - 챗: 대화
 - 컨텍스트: 챗의 모음
 - 대화: 챗과 컨텍스트의 모음, 2개월간 저장
 - 아카이브: 영구 저장된 대화
 - 메모리: 아카이브 중 주요 내용을 항상 활용할 수 있도록 준비

II. LLM 활용과 프롬프팅

- 메모리

- 사용자와의 이전 대화를 기억하고,이후 대화에 지속적으로 반영하는 기능
- 반복 설명 없이, 개인화된 대화 가능
- 메모리 관리 방법
 - 좌측 메뉴 > 프로필 > "메모리" 메뉴 켜기 / 끄기
- 개별 정보 삭제 가능 + 각 대화별로도 비활성화 가능
- 민감한 정보(비밀번호, 개인정보 등) 는 저장하지 않음
- 챗GPT가 기억한 내용은 수동으로 확인 및 수정 가능
 - 메모리는 대화 중 자동으로 생성되며, GPT가 "이 내용을 기억해도 될까요?"라고 확인

II. LLM 활용과 프롬프팅

- Technique #1 – 데이터의 생성과 분석

10명의 사원의 영업실적 데이터를 생성해보세요. 데이터에는 매출액 변수가 있고 평균 1억원을 중심으로 생성하고, 지출액 변수가 있고 평균 5천만원을 중심으로 생성하고, 매출액과 지출액의 차이를 영업이익 변수로 생성하세요.

이 데이터의 모든 변수에 대해, 주요 통계량을 분석하고, 결과를 해석하세요.

앞서 생성한 10명 영업사원의 군집을 발견하고 결과를 설명해보세요.
지출액에 따른 영업이익을 예측하는 모델을 만들고 설명하세요.

II. LLM 활용과 프롬프팅

- Technique #2 -설문 개발 및 설문 결과 (또는 엑셀 표) 처리 및 분석

대박전자의 현재 고객들에게 스마트워치의 개선을 위해서 필요한 부분을 설문하려고 합니다. 설문 문항을 5점척도를 사용해 10개 작성하시오.

아래 설문 결과는 설문에 대한 결과입니다. 각 행에는 하나의 응답이, 각 변수는 설문 문항에 대한 답변이 순서대로 있습니다.

<설문결과>

5,4,2,3,1,5,4,4,4,3

4,3,4,4,5,2,3,4,2,4

5,5,5,5,5,4,4,4,3,5

3,4,4,3,5,5,4,3,4,5

5,5,2,3,4,5,4,4,4,3

4,3,4,4,3,2,3,2,2,4

5,5,3,5,5,4,5,4,3,5

5,4,5,3,5,5,4,3,4,5

3,4,4,3,5,5,4,3,4,5

5,5,2,3,4,5,4,4,4,3

</설문결과>

엑셀의 내용을 복사해서 붙이기 또는 엑셀 업로드

이 설문을 분석하고 해석해서 보고서를 만드세요.

II. LLM 활용과 프롬프팅

• Technique #3 – 텍스트 요약과 정리

<https://ko.wikipedia.org/wiki/%EC%9D%B8%EA%B3%B5%EC%A7%80%EB%8A%A5의 내용을 300글자 미만으로 요약해줘>



“초기 인공지능 연구에 대한 대표적인 정의는 다트머스 회의에서 존 매카시가 제안한 것으로 "기계를 인간 행동의 지식에서와 같이 행동하게 만드는 것"이다. 그러나 이 정의는 범용인공지능(AGI, 강한 인공지능)에 대한 고려를 하지 못한 것 같다. 인공지능의 또다른 정의는 인공적인 장치들이 가지는 지능이다. 거의 대부분 정의들이 인간처럼 사고하는 시스템, 인간처럼 행동하는 시스템, 이성적으로 사고하는 시스템 그리고 이성적으로 행동하는 시스템이라는 4개의 분류로 분류된다” 이 글을 개조식으로 표 형태로 정리해줘

II. LLM 활용과 프롬프팅

• Technique #4 – 비즈니스 분석

정성적 분석

1. SWOT분석
2. PEST 분석
3. AARRR 분석
4. Action Plan 수립
5. 그 외

정량적 분석

1. DEA
2. 통계분석
3. 머신러닝
4. 그 외

대박전자의 새 스마트폰 대박폰을 출시하려고 합니다. 대박폰은 가장 큰 화면과 가장 큰 배터리, 가장 무거운 무게, 그렇지만 저렴한 가격으로 준비되었습니다.

1. 이 제품에 대한 swot분석을 해서 표가 포함된 보고서로 정리하세요
2. 대박전자에서 대박폰에 대한 왕대박SNS 를 운영하려고 합니다. 40대를 타겟으로 하여, 이에 대한 AARRR을 수립하여서 표로 정리하세요
3. 대박전자의 온라인 SNS, 왕대박,의 개발과 런치를 위한 액션플랜을 세우시오. 25명의 인력과 6개월의 시간, 1억원의 예산이 있습니다.sns에서는 대박전자의 신기술을 체험하고 확산하고 홍보하는 것을 목표로 합니다. 1개월마다 성과 점검을 하며, 성과는 직원들 인센티브에 연동합니다.

II. LLM 활용과 프롬프팅

- Technique #5 – 바이브코딩

- 개발하려는 기능을 제시한 후 코드 생성
- 파이썬, C, Java 등 다양한 언어로 코드 생성

파이썬으로 계산기 만드는 코드

키, 몸무게, 학년 정보를 갖는 10명의 중학생의 데이터를 생성, 가장 적절한 시각화를 하도록 d3.js를 이용해 하나의 html 을 만드시오

위의 시각화를 인터랙티브하게 바꾸도록 d3.js를 이용해 하나의 html을 만드시오

II. LLM 활용과 프롬프팅

• Technique #5 – 애플리케이션 개발

코랩에서 웹캠으로 사진찍은 후, 결과를 저장하고, 이미지 크기를 변환하는 코드를 만드세요

코랩(Colab)에서 웹캠으로 사진을 찍고, 그 이미지를 저장한 후 크기를 변환하는 전체 코드를 아래에 제공합니다. 이 코드는 `cv2` (OpenCV)를 사용하며, 주피터 환경에 맞춰 구성되어 있습니다.

✓ 1. 필요한 라이브러리 설치 및 불러오기

```
python

!pip install opencv-python-headless --quiet

import cv2
from google.colab.patches import cv2_imshow
from google.colab import output
from IPython.display import display, Javascript
from IPython.display import Image
import io
import PIL.Image
import numpy as np
```

✓ 2. 웹캠 촬영 함수 (자바스크립트)

```
python

def take_photo(filename='photo.jpg', quality=0.8):
    js = Javascript('''
        async function takePhoto(quality) {
            const div = document.createElement('div');
            const capture = document.createElement('button');
            capture.textContent = '📷 사진 찍기';
            div.appendChild(capture);

            const video = document.createElement('video');
            video.style.display = 'block';
            div.appendChild(video);

            document.body.appendChild(div);

            const stream = await navigator.mediaDevices.getUserMedia({video: true});

            video.srcObject = stream;
            await video.play();

            // 사진 찍기 버튼 클릭 대기
            await new Promise((resolve) => capture.onclick = resolve);
        }
    ''')
```

✓ 3. 사진 촬영 및 표시

```
python

import base64

filename = take_photo()
print(f"저장된 파일 이름: {filename}")

# 이미지 불러오기
img = cv2.imread(filename)
cv2_imshow(img)
```

✓ 4. 이미지 크기 변환 (예: 300x300)

```
python

# 크기 조정
resized = cv2.resize(img, (300, 300))

# 저장
cv2.imwrite("resized_photo.jpg", resized)
print("크기 변경된 이미지 저장 완료: resized_photo.jpg")

# 확인
cv2_imshow(resized)
```

III. Work, Rag, Finetuning

- 챗GPT Work모드: Agent, Agentic AI

- 복잡한 업무를 목표부터 결과물까지 수행하는 AI 에이전트

- 단순 질의응답이 아니라 여러 단계의 업무를 계획하고 순차적으로 수행
 - 작업 중 사용자가 방향 수정 및 주요 작업 승인 가능
 - 정보 조사 및 분석: 웹, 첨부파일, 프로젝트 자료 등을 활용하여 필요한 정보 수집, 수집된 자료를 분석·요약하여 업무 결과로 연결
 - 앱/파일 연계 작업: 연결된 앱과 파일을 넘나들며 필요한 자료를 활용, 예를 들어 Google Docs/Sheets/Slides 등과 연계 가능
 - 완성된 결과물 생성: 문서, 보고서, 스프레드시트, 프레젠테이션, 분석자료, Site 등을 직접 생성
 - 기존 파일이나 템플릿을 기반으로 수정/확장 가능
 - 반복/지속 업무 자동화
 - 한 번 실행하는 작업뿐 아니라 일정 기반 반복 실행
 - 특정 변화 발생 여부 모니터링 지원

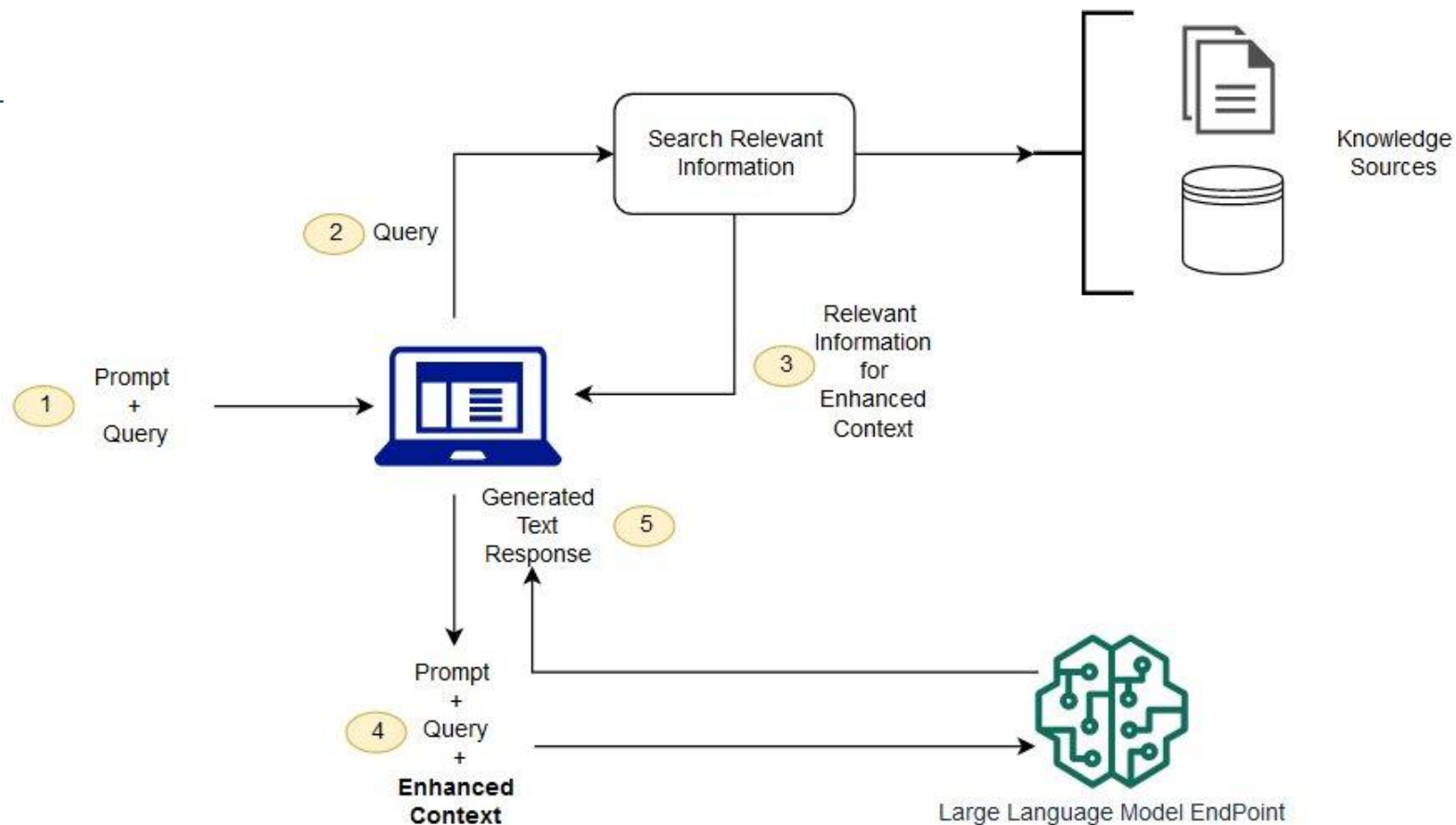
III. Work, Rag, Finetuning

- RAG: 검색 증강 생성
 - Retrieval-Augmented Generation
 - Hallucination을 줄이는 비용 효율적인 접근 방식
 - LLM의 기능을 특정 도메인이나 특정 기관의 내부 지식 기반으로 확장
 - LLM 외부의 데이터를 활용해서 보다 정확한 답변을 찾는 방안
 - 모형 학습을 다시 할 필요는 없음

III. Work, Rag, Finetuning

- RAG 과정

- LLM 학습 데이터 외의 데이터인 외부 데이터 생성 후 임베딩을 통해 벡터 DB저장
- 질문에 대한 관련 정보 검색
- 외부 데이터 업데이트: 실시간 및 배치 방식



III. Work, Rag, Finetuning

- **RAG: 검색 증강 생성**
 - 1단계:
 - 사용자 질문 입력사용자가 질문과 함께 프롬프트 입력
 - 예: "2026년 전기차 보조금 정책은 어떻게 바뀌었는지 알려줘"
 - 2단계:
 - 질문 → 웹 검색 쿼리로 변환
 - 핵심 키워드 중심의 간결한 쿼리 생성예: "2025년 한국 전기차 보조금 변경 내용"
 - 3단계:
 - 웹 검색 및 정보 수집정부/언론 등 신뢰 가능한 출처 탐색
 - 주요 문장 추출 및 요약예: "보조금 600만 원 → 500만 원 축소, 고가 차량 제외"

- RAG: 검색 증강 생성
 - 4단계:
 - 확장된 문맥 생성 (Enhanced Context)
 - 외부 정보 + 프롬프트 + 사용자 질문 결합 → LLM 입력용 고도화된 문맥 생성
 - 5단계:
 - LLM이 최종 응답 생성요약, 설명, 근거 등 사용자 맞춤 응답 출력
 - 예: "보조금이 500만 원으로 축소되며, 고가 차량은 제외..."

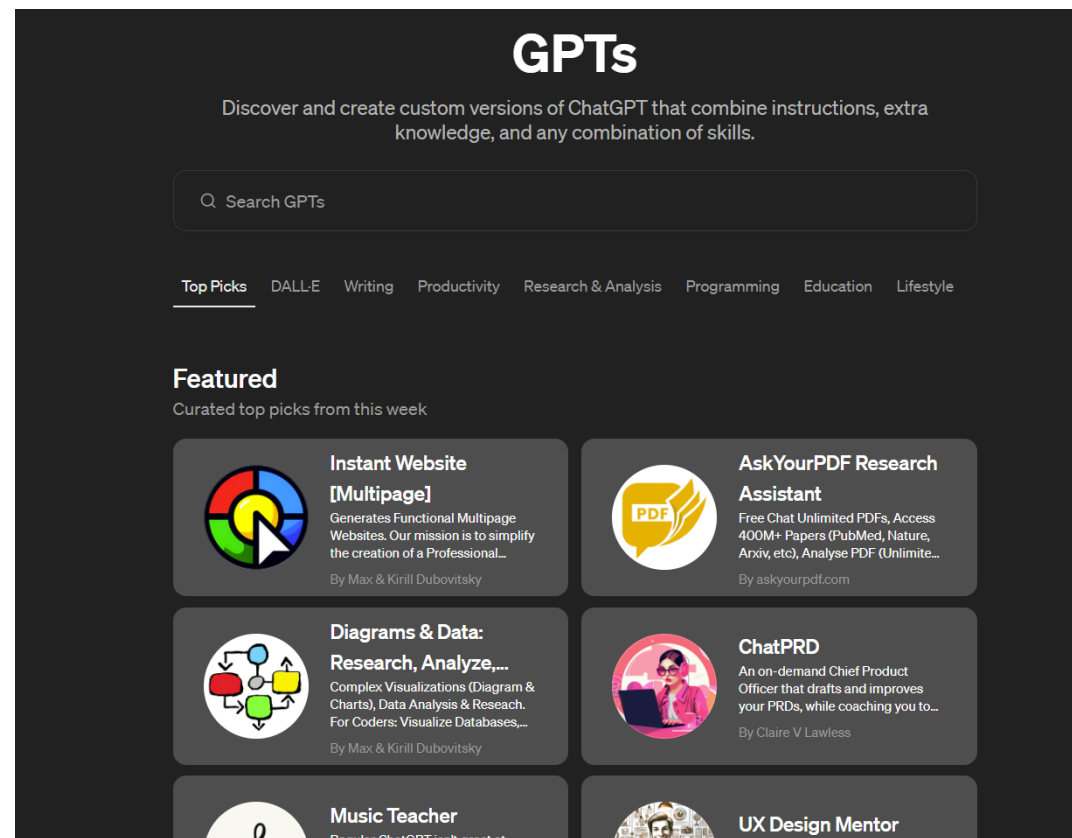
III. Work, Rag, Finetuning

- Fine Tuning
 - 기존 GPT 모델에 특정 데이터셋을 추가 학습시켜 도메인 특화 모델을 만드는 방법
 - GPT의 기본 성능 + 맞춤형 지식/스타일 내재화
 - Fine Tuning의 특징
 - 일회성 응답이 아닌 지속적으로 반영되는 학습 결과
 - 다음을 모델 자체에 자동 학습
 - 도메인 지식
 - 브랜드 어조
 - 업무 스타일 등

III. Work, Rag, Finetuning

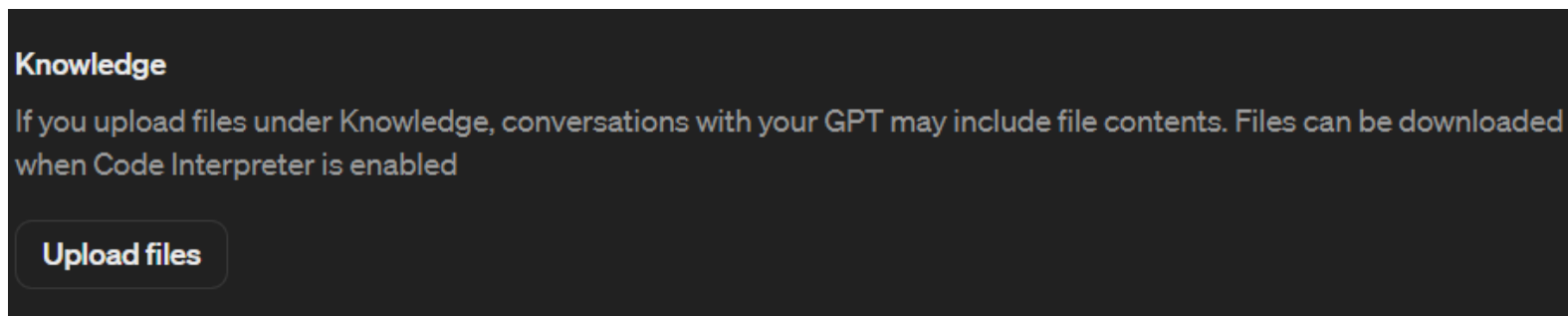
- GPT

- GPT: Customize된 챗GPT를 의미
 - instruction: 제약사항, 가이드라인, 설명, 개인화 등을 확인

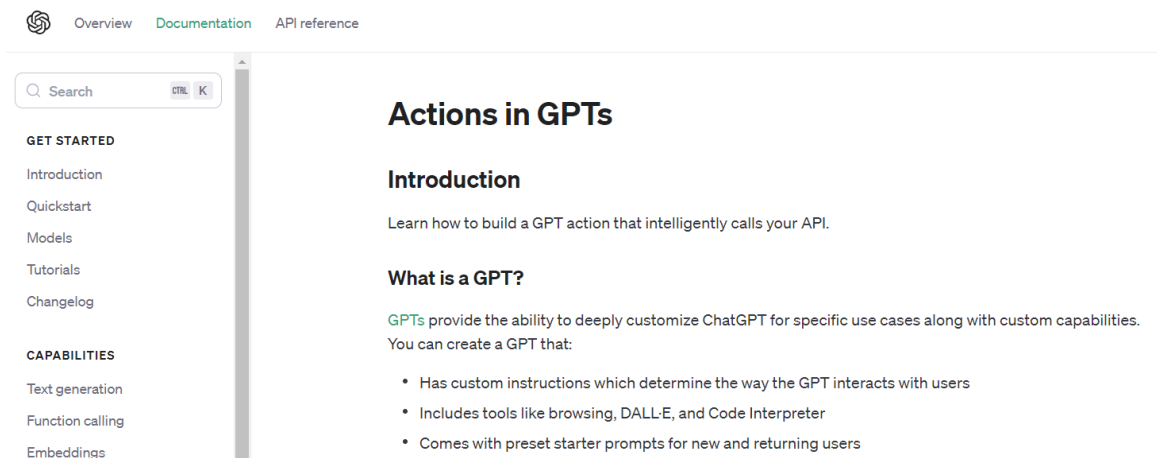


III. Work, Rag, Finetuning

- GPT 설정
 - Knowledge에 파일 업로드



- 외부 API의 활용



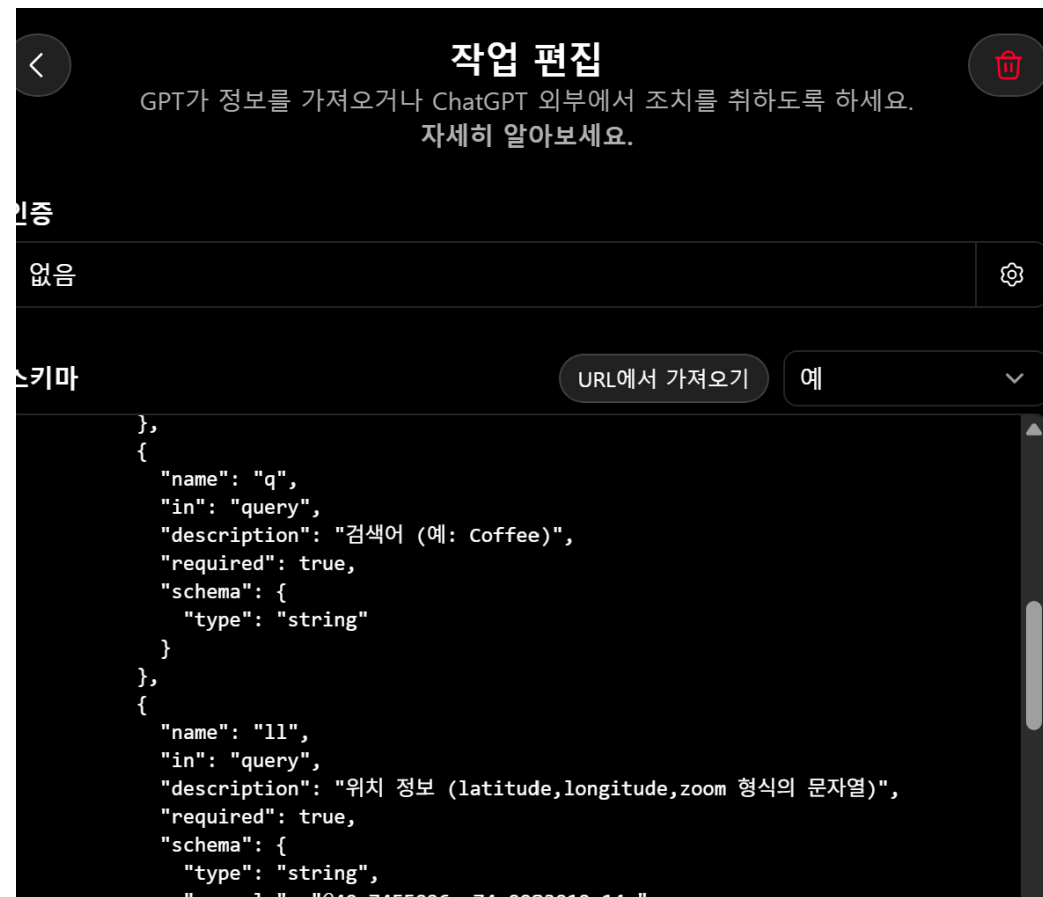
III. Work, Rag, Finetuning

- GPT 생성

- Actions

- API? API는 컴퓨터나 컴퓨터 프로그램 사이의 연결이다. 일종의 소프트웨어 인터페이스이며 다른 종류의 소프트웨어에 서비스를 제공
 - YAML, JSON, 그 외 양식 지원

- 참고: <https://platform.openai.com/docs/actions/introduction>



IV. Vibe coding 소개

- Vibe 코딩
 - Vibe 코딩(Vibe Coding)이란?
 - AI 중심 소프트웨어 개발 방식
 - Andrej Karpathy(OpenAI 공동 창업자)가 제안
 - 기존의 명시적 코딩 → 자연어 지시 + AI 응답
 - Vibe 코딩의 핵심 철학
 - “보고 말하고 실행하고 복사하여 붙여넣는 일련의 행위”
 - 기술/언어 제약 없이, 아이디어 → 결과물로 빠르게 전환



IV. Vibe coding 소개

- Vibe 코딩

- 직관적 개발

- 자연어로 명령, 자동 코드 생성

- "캔버스에서 장애물을 피하는 게임 만들어줘" → 간단한 게임 완성

- 빠른 프로토타입 제작: MVP

- 비전문가도 AI로 직접 프로토타입 제작 가능

- 개발 속도 향상, 기술 장벽 완화

- Vibe 코딩 결과물 ?= Production System

- AI가 생성한 코드는 복잡한 상황에 취약, 전문 개발자의 코드 검토 및 수정이 필요

- 아이디어 우선의 함정: 실현 가능성보다 컨셉에 집중될 수 있음

- 개발자의 영역: Vibe 코딩만으로는 모든 문제 해결 불가

- » 복잡한 알고리즘, 성능 최적화 등: 숙련된 개발자의 영역

IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 1

- 코드 인터프리터

- 챗GPT에서 바이브코딩

- 코드 명령어 입력: "데이터를 분석하여 매출 증가율을 시각화하는 파이썬 코드를 작성해줘."

- 코드 생성: 챗GPT는 명령어를 해석하여 파이썬 코드를 자동으로 생성

```
python 📄 복사 ✎ 편집  
  
import pandas as pd  
import matplotlib.pyplot as plt  
  
# 가상 데이터 생성  
data = {'월': ['1월', '2월', '3월', '4월'], '매출': [100, 150, 200, 300]}  
df = pd.DataFrame(data)  
  
# 시각화  
plt.plot(df['월'], df['매출'])  
plt.title('매출 증가율')  
plt.show()
```

--- *** ---

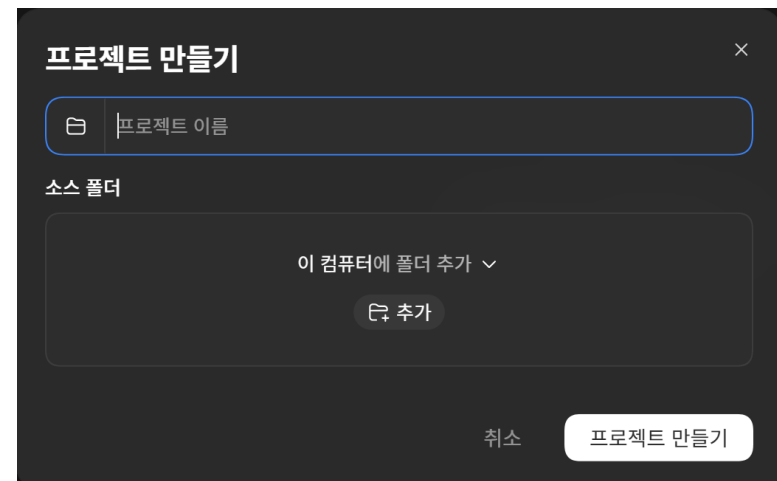
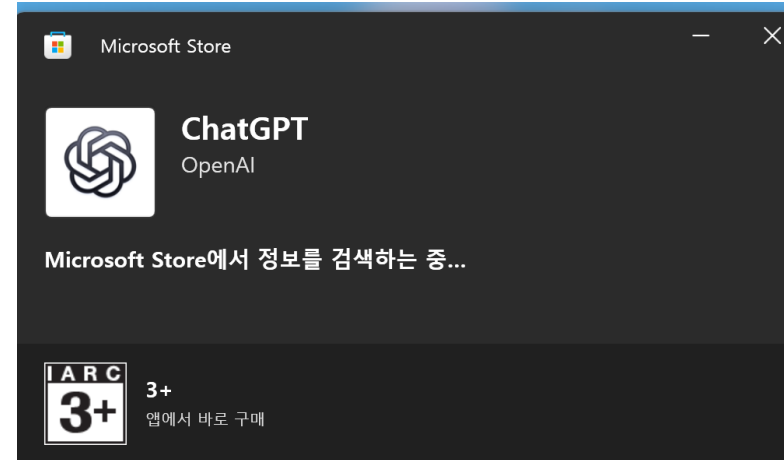
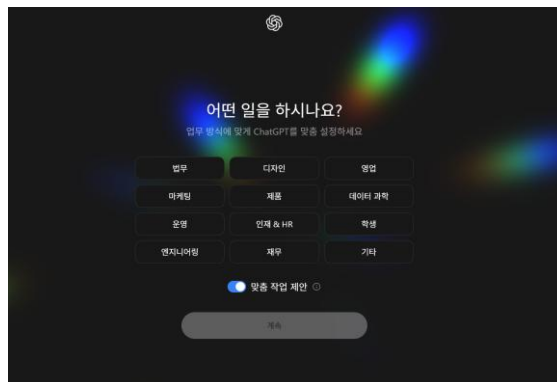
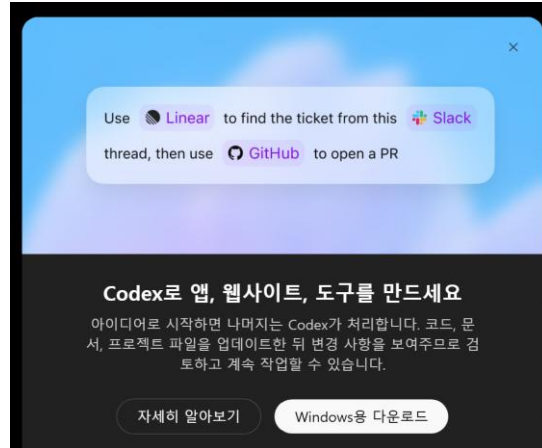
IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 2
 - 자연어 명령을 코드로 변환하여 실행: "명령 기반 코드 생성"을 실현
 - **Codex**
 - 개발 Agent
 - CLI 버전 또는 IDE Extension으로 사용
 - 로컬 개발환경에서 활용이 필요
 - 여러 파일을 동시에 접근하여 프로젝트 완성

IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 2

- Codex: 설치 전 사내 보안 준수여부 확인 / 800MB

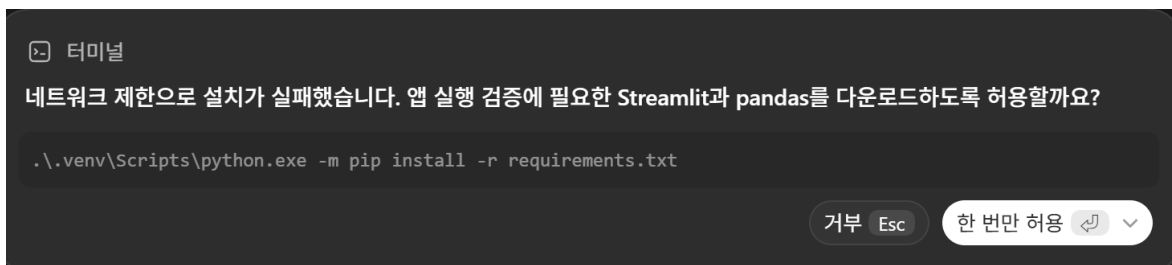
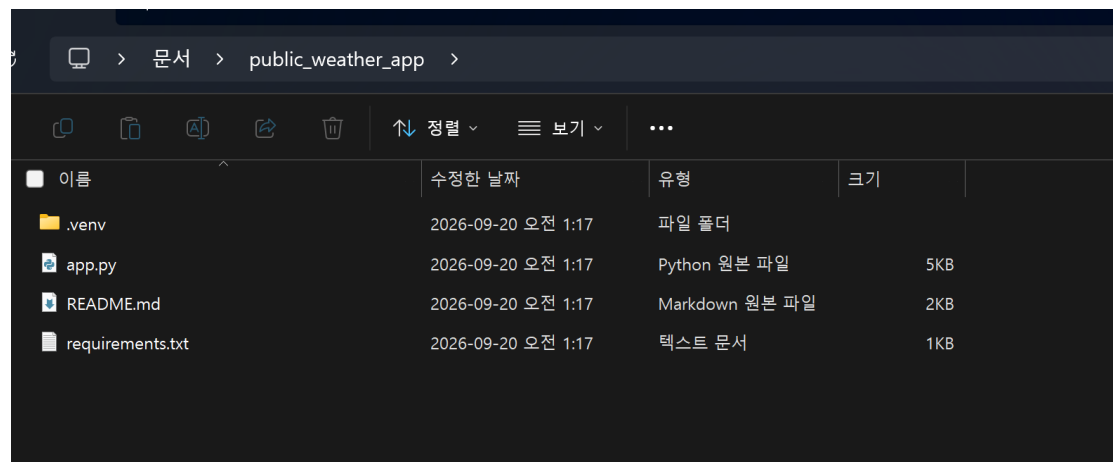
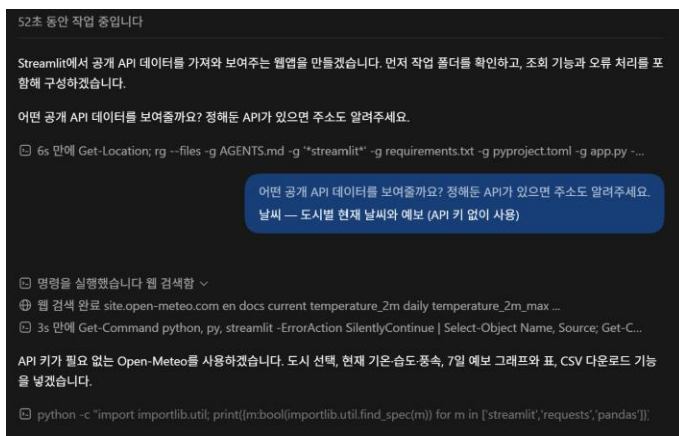


IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 2

- Codex

Streamlit으로 웹앱 제작, 공개된 api에서 값을 가져와서 보여주는 기능



IV. Vibe coding 소개

• 챗GPT의 Vibe 코딩 2

— Codex

TS 전에 `.\venv\Scripts\python.exe check_app.py` 실행

도시 변경, 지표 4개 표시, 7일 예보 표, 연결 오류 안내 검사를 통과했습니다. 마지막으로 실제 API 응답을 확인하고 로컬 실행을 준비하겠습니다.

```
.\venv\Scripts\python.exe -c "from streamlit.testing.v1 import AppTest; a=AppTest.from_file('app.py').run(timeout
```

Streamlit 날씨 앱을 만들었습니다.

- 국내외 10개 도시 선택
- 현재 기온·체감온도·습도·풍속
- 7일 예보 그래프와 표
- CSV 다운로드, 새로고침, 오류 안내

 Open-Meteo API 를 사용하며, 실제 데이터 조회와 기능 검사를 통과했습니다. 패키지 설치도 완료했습니다.

PowerShell에서 실행하세요:

```
</> powershell
```

```
cd C:\Users\user\Documents\public_weather_app
.\venv\Scripts\python.exe -m streamlit run app.py
```

실행 후  웹앱 열기

 소스 코드 ·  실행 안내

 웹 미리보기
웹사이트

다음에서 열기 ▾

 파일 5개를 편집했습니다
+160 -3

실행 취소 ↶ 리뷰

조회 설정

도시

서울 ▾

새로고침

조회 결과는 10분간 캐시됩니다. 새로고침하면 다시 조회합니다.

도시 날씨

오늘의 날씨와 앞으로 7일의 예보를 확인하세요.

서울 · 현재 날씨

기준 시각: 2026-09-20 01:15 · Asia/Seoul | 조회 시각: 2026-09-19 16:23 UTC

기온	체감 온도	습도	풍속
18.9 °C	20.0 °C	80 %	1.43 m/s

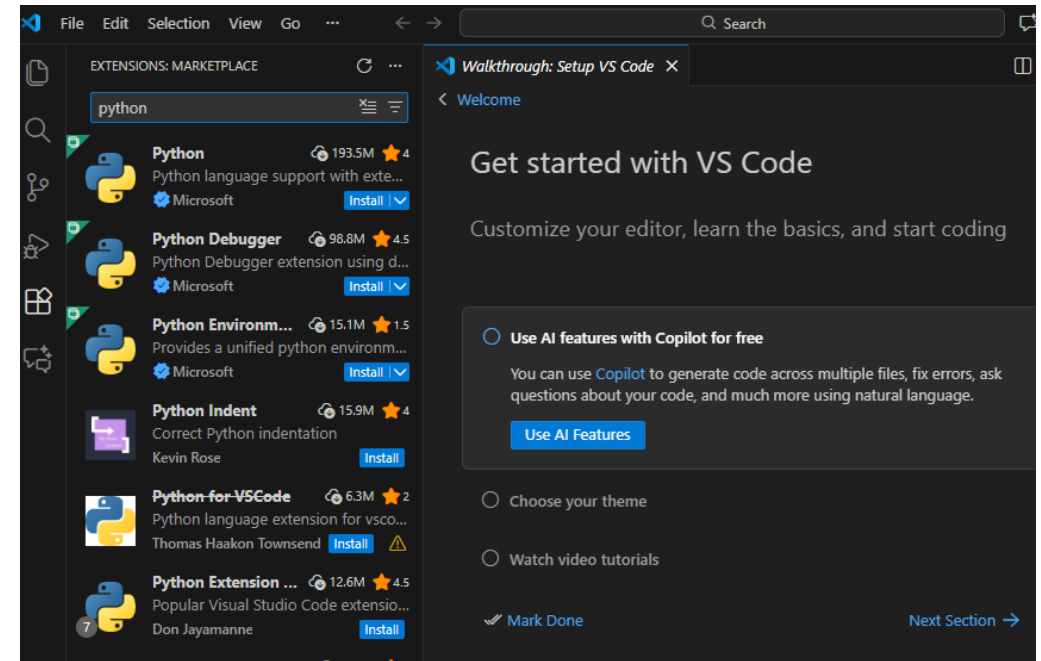
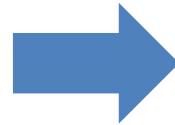
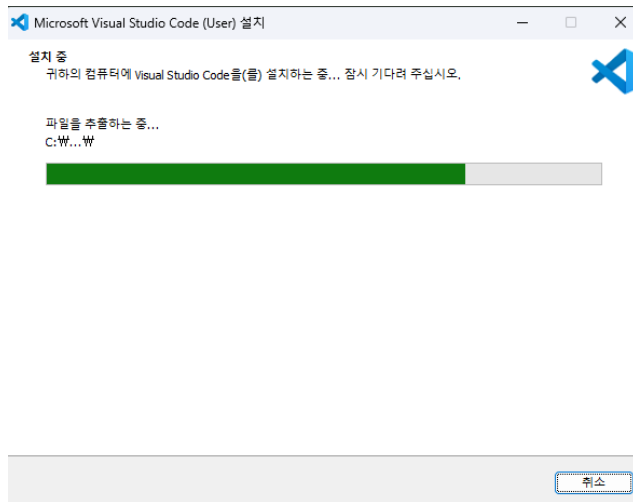
7일 예보



IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 3: VS Code + LLM

- VS Code 설치 -> MS 제공 Python 설치 -> Extension에서 챗gpt 또는 copilot, copilot chat 설치-> Github계정 연동으로 copilot 사용



IV. Vibe coding 소개

The image shows a screenshot of the Visual Studio Code (VS Code) interface. On the left, the 'EXTENSIONS: MARKETPLACE' sidebar is visible, displaying a list of Python-related extensions. The main editor area shows the 'Walkthrough: Setup VS Code' window with the title 'Get started with VS Code'. A modal dialog is overlaid in the center, titled 'Sign in to use AI Features', which prompts the user to sign in using GitHub, Google, Apple, or GHE.com. To the right, the 'EXTENSIONS: MARKETPLACE' sidebar is also visible, showing a list of Python extensions. Below the sign-in modal, there is a blue arrow pointing right and a blue arrow pointing left.

EXTENSIONS: MARKETPLACE

Extension Name	Author	Downloads	Rating	Install Button
Python	Microsoft	193.5M	4.5	Install
Python Debugger	Microsoft	98.8M	4.5	Install
Python Environm...	Microsoft	15.1M	1.5	Install
Python Indent	Kevin Rose	15.9M	4	Install
Python for VSCode	Thomas Haakon Townsend	6.3M	2	Install
Python Extension ...	Don Jayamanne	12.6M	4.5	Install

Walkthrough: Setup VS Code

Get started with VS Code

Customize your editor, learn the basics, and start coding

☐ Use AI features with Copilot for free

You can use Copilot to generate code across multi questions about your code, and much more using

[Use AI Features](#)

☐ Choose your theme

☐ Watch video tutorials

[Mark Done](#)

Sign in to use AI Features

[Continue with GitHub](#)

[Continue with Google](#)

[Continue with Apple](#)

[Continue with GHE.com](#)

[Skip for now](#)

By continuing, you agree to GitHub's [Terms](#) and [Privacy Statement](#). GitHub Copilot may show [public code](#) suggestions and use your data to improve the product. You can change these [settings](#) anytime.

EXTENSIONS: MARKETPLACE

Extension Name	Author	Downloads	Rating	Install Button
Python	Microsoft	193.5M	4.5	Install
Python Debugger	Microsoft	98.8M	4.5	Install
Python Environm...	Microsoft	15.1M	1.5	Install
Python Indent	Kevin Rose	15.9M	4	Install
Python for VSCode	Thomas Haakon Townsend	6.3M	2	Install
Python Extension ...	Don Jayamanne	12.6M	4.5	Install
Python Environmen...	Don Jayamanne	12.7M	4	Install
autoDocstring - Pyth...	Nils Werner	15M	5	Install

Walkthrough: Setup VS Code

[Go Back](#)

GitHub Copilot

Your AI pair programmer to write code faster and smarter

☐ Chat about your code

Ask Copilot programming questions or get help with your code using `@workspace`.

Type `@` to see all available chat participants that you can chat with directly, each with their own expertise.

[Chat with Copilot](#)

☐ Make changes using natural language

☐ AI-suggested inline suggestions

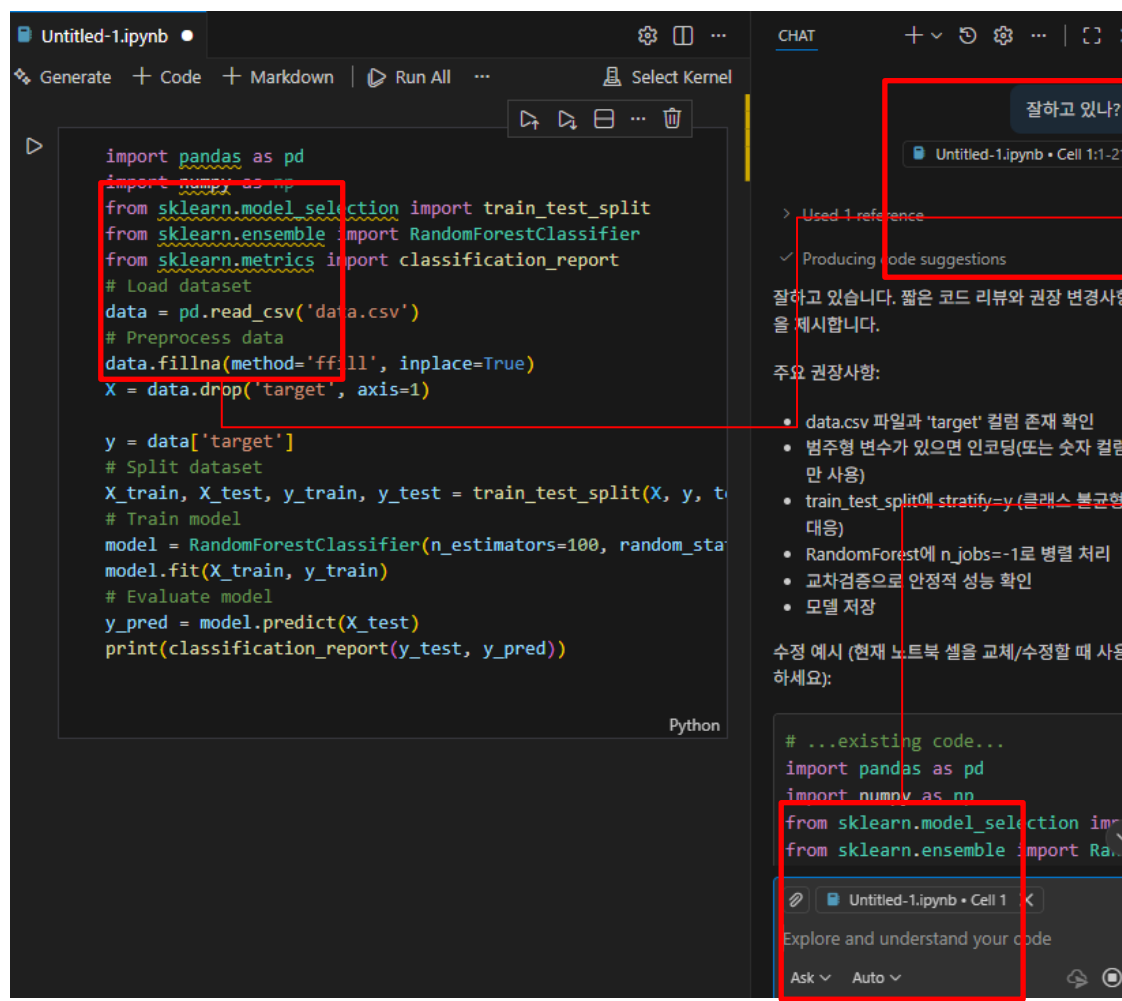
☐ Use natural language in your files

☐ Look out for smart actions

IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 3: VS Code + LLM

- New File -> Python 또는 Jupyter Notebook 선택



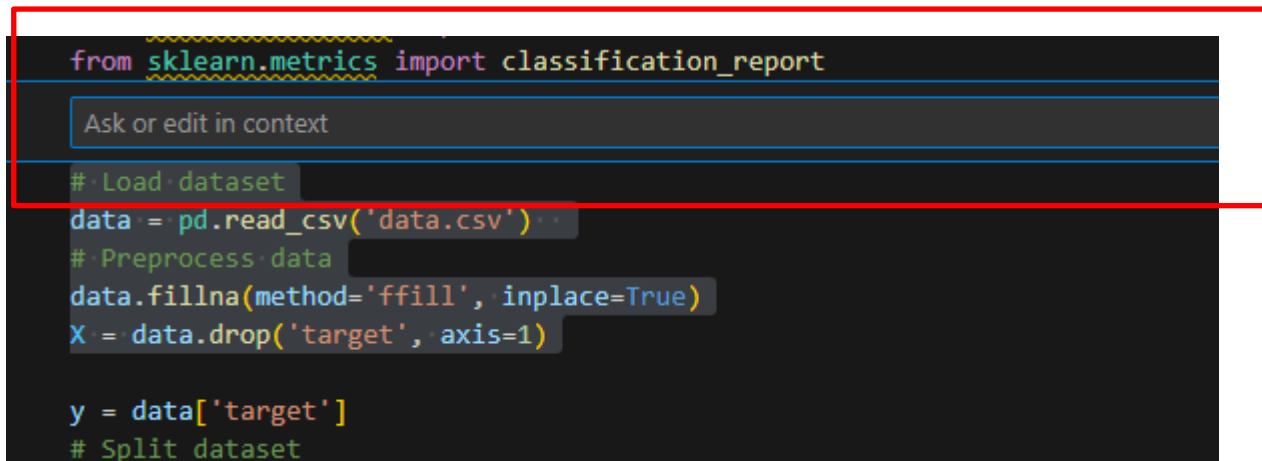
- 주석에 한글로 코드에 대한 설명->자동완성

- 채팅 창에 프롬프트로 코드 요청->이후 반영

- Ask, Edit, Agent 모드

IV. Vibe coding 소개

- 챗GPT의 Vibe 코딩 3: VS Code + LLM



```
from sklearn.metrics import classification_report
```

Ask or edit in context

```
# Load dataset
data = pd.read_csv('data.csv')
# Preprocess data
data.fillna(method='ffill', inplace=True)
X = data.drop('target', axis=1)

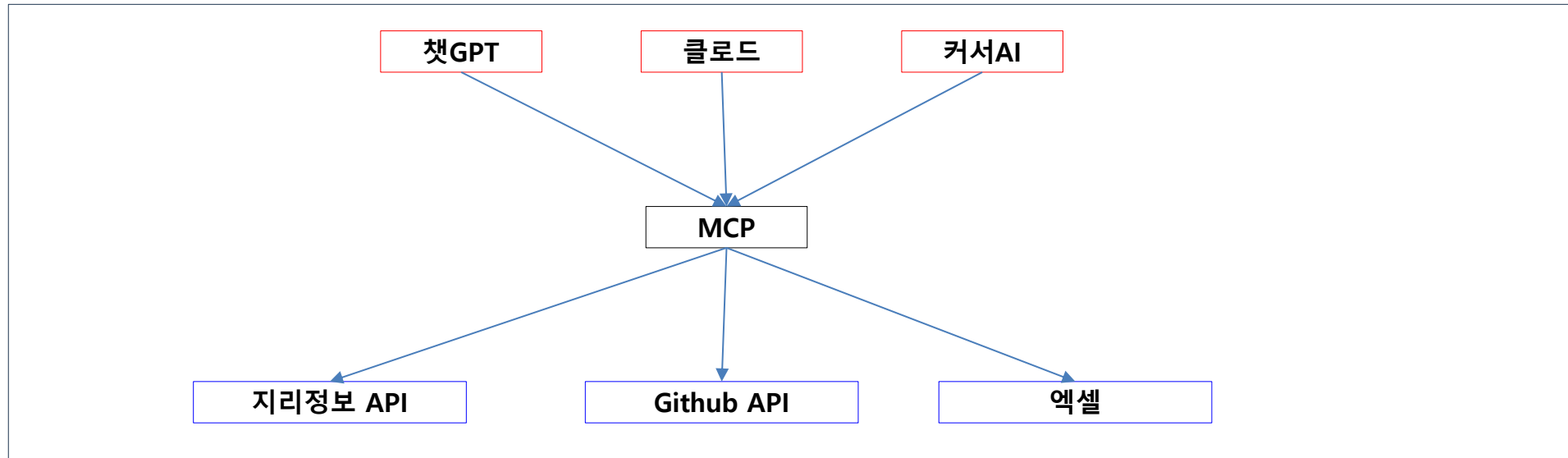
y = data['target']
# Split dataset
```

- 영역 지정 후 CTRL+I
- 코드 리팩토링 또는 설명 요청

- MCP
 - AI agent: "LLM+도구"로 실제 행동을 수행시키는 기술
 - **Model Context Protocol**: LLM이 외부의 다양한 도구(tool)와 구조화된 방식으로 인터랙션하는 프로토콜
 - 도구(tool): LLM 외부의 데이터/정보를 다룰 수 있는 애플리케이션/API 등 LLM 외부의 기능들
 - '범용 어댑터' : Tool이 MCP를 따르면, 클로드 데스크톱, 커서AI에서도 연동
 - 기존 Open AI의 Function calling, 클로드의 tool_use 메시지구조, Langchain 의 Agent 기반 등 다양한 방식이 존재

V. MCP 소개

- MCP
 - 모델과 도구의 연결



- MCP

- MCP 서버: tool, 스미더리 등에서 공용으로 호스팅되어 제공되기도 함.
- MCP 클라이언트: MCP서버에 요청 보내고 결과를 받아 호스트에 전달, 예: 커서 내부 LLM 호출 모듈, 클로드 데스크톱의 MCP 요청 처리기 등
- MCP 호스트: 클로드 데스크톱, 커서, 윈드서프 등 LLM 기반 App, LLM
- 도구 정의 및 불러오기
 - 각 도구는 독립된 MCP 서버로 구현: 코드 실행기, 파일 분석기, 이메일 발송기 등
 - JSON으로 도구 기능 표현
 - 클라이언트는 LLM 으로 JSON을 전달
 - LLM 도구 선택 및 호출

- 주요 MCP 클라이언트

- Claude Desktop: 사용자가 직접 MCP 서버 연결 (JSON 도구 목록 인식)
- 커서 AI: AI기반 코드 편집기, VS Code를 기반으로 GPT 또는 클로드 모델 활용
- 윈드서프(Windsurf)
 - 워크 플로우 중심의 AI 코드 에이전트
 - 여러 MCP 도구를 조합해 자동화된 작업 흐름 구성
- Smithery: 여러 MCP 서버들의 집합, 워크플로우 생성

- MCP 통신 방식
 - Stdio(표준 입출력)
 - MCP 서버가 터미널에서 실행
 - Stdio로 서버와 클라이언트가 데이터 주고받음
 - 서버와 클라이언트가 동일한 컴퓨터, 빠른 실행
 - SSE(Server Sent Events)
 - HTTP 기반 스트리밍으로 데이터 송수신, 웹 서버 형태의 MCP서버에 적용
 - HTTP Post 방식
 - 다중 클라이언트 및 세션 관리, 실시간 응답 및 스트리밍에 유리

- **MCP 구성 요소**
 - 도구: @mcp.tool()로 등록
 - 리소스: @mcp.resource()로 등록, 참조용 데이터 또는 문맥
 - 프롬프트: @mcp.prompt()로 등록
- **클라이언트에 도구 등록하는 다양한 방법**
 - 모든 플랫폼: 파이썬코드로 구현된 도구를 직접 실행
 - 커서 등: Command 호출 등록
 - 커서: 정적 등록, mcp.json으로 설정하여 클라이언트 자동 등록
 - GUI 등록: 클로드 데스크톱 개발자 모드
 - 원격서버 연결: 웹 환경 전용, 웹 서버 필요

V. MCP 소개

- MCP 단계별 이해

- ① 프롬프트로 질문 또는 명령 입력
- ② LLM이 입력 분석 후 외부 정보 필요여부 판단
- ③ 필요 시 MCP 요청 생성: 외부 도구의 호출 준비
- ④ MCP 서버에서 요청 수신
- ⑤ MCP 서버가 요청된 도구(기능, 실행 등)를 실행
- ⑥ 클라이언트로 결과 반환
- ⑦ LLM이 MCP 결과를 포함해 자연어로 답변 생성

- **챗GPT + MCP** : AI가 파일/DB/GitHub/업무 시스템 등 외부 세계와 연결되는 표준 인터페이스
 - ChatGPT
 - 외부 앱·데이터 소스를 MCP 기반으로 연결 가능
 - 연결된 시스템의 정보 검색·조회 및 일부 작업 수행
 - Codex
 - MCP 서버를 연결하여 개발 환경 및 외부 도구 활용
 - 로컬 개발환경과의 연계에 특히 유용
 - OpenAI API
 - Responses API / Agents 환경에서 MCP 도구 사용 가능
 - 자체 서비스 및 연구 시스템과의 통합 가능

- **챗GPT 플러그인**
 - Plugin = 사용자가 설치·사용하는 기능 패키지
 - App = Google Drive, DB, 사내 시스템 등 외부 서비스 연결
 - MCP = AI와 외부 App/도구를 연결하는 표준 프로토콜
 - 흐름: ChatGPT → Plugin → App → MCP → 외부 시스템
 - 모든 플러그인이 MCP를 쓰지는 않음
 - MCP 기반 App이 플러그인에 포함될 수 있음

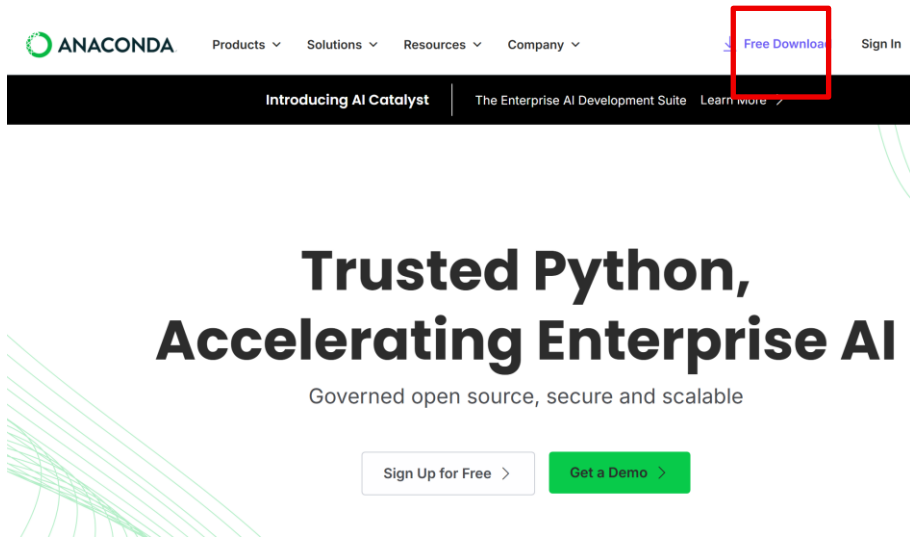
- AI :
 - **챗GPT(Free or Pro)**
 - 커서(Free)
- 로컬 환경:
 - 커서
 - **Vscode+Copilot(Free)+python**
 - 사용 중인 IDE

다양한 조합이 가능

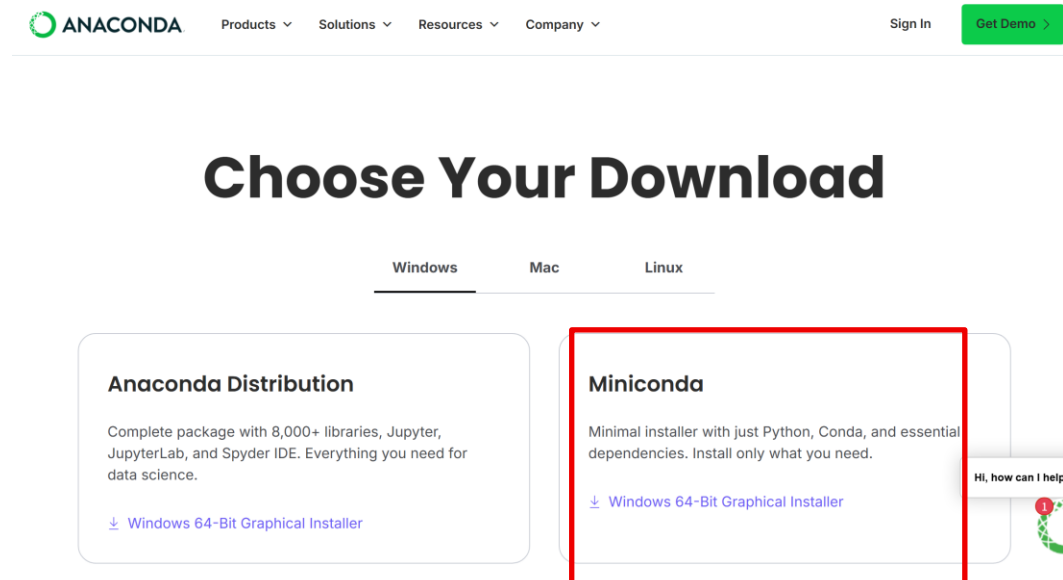
VI. LLM 실습

- 참고: Python 환경

- 미니콘다 설치: <https://www.anaconda.com> -> Free Download -> Miniconda 다운로드 후 설치

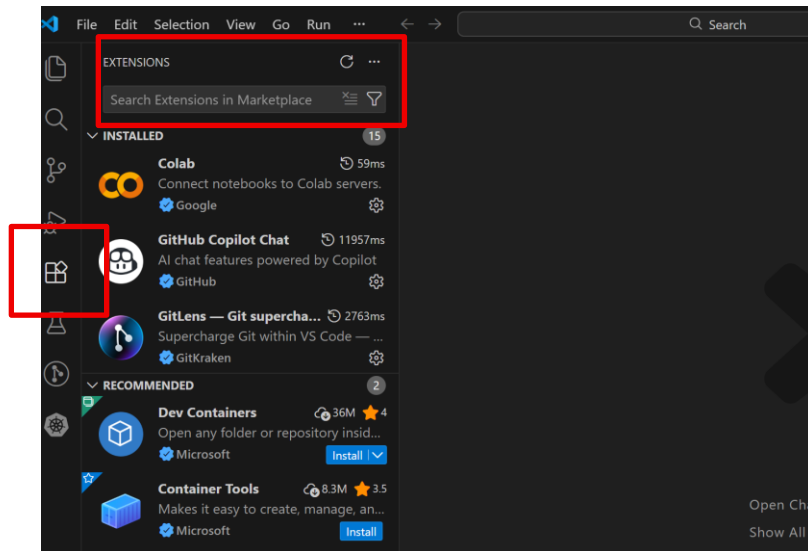


Registration skip으로 다운로드 가능



- 참고: VS코드 설치

- <https://code.visualstudio.com/> 설치 파일 다운로드 후 설치
- Extension설치: 하단 그림 내 Extension 클릭 후, 검색 창에 아래 Extension 들을 검색하여 Install



- Github Copilot (Publisher: Github)
- Python (Publisher: Microsoft)
- Jupyter (Publisher: Microsoft)

VS코드에서 ipynb 파일을 열고 저장 후 우측 상단 kernel 클릭, 생성한 환경 선택

VI. LLM 실습

• 챗GPT에서 실행

분석결과

- 장애 발생 전(2028경) 전차선 전압 26.95[kV], 전차선 전류 70.8[A] 발생
- 장애 발생 시(2029경) 전차선 전압 27.31[kV], 전차선 전류 1,869.7[A] 발생
- 절연과피로 인한 저차 고조파(2~13차) 발생 확인
 - (평상시) 47차, 49차 고조파 전압 / (사고시) 2~13차 고조파 전압

측정일(기)	'26. 6. 25.(목) 20:29 ¹²	'26. 6. 25.(목) 20:28 ¹²	'26. 6. 24.(수) 월간	'26. 6. 24.(수) 월간
측정구간	동해SS-간인SP(F3)	동해SS-간인SP(F3)	동해SS-간인SP(F3)	동해SS-동화SP(F5)
전차선 전압	27.31[kV]	26.95[kV]	27.44[kV]	27.41[kV]
전류	1,869.7[A]	70.8[A]	210.7[A]	173.3[A]



260625_2000_2100_동해SS F3_전력...
스프레드시트

첨부된 자료로, 그림같은 결과가 나오려면 어떤 분석을 해 볼까?

- ☑ 검사하고 API 빠른 시작 가이드 검토
- ☑ 워크북 시트 및 데이터 구조 검사 완료
- ☑ 엑셀 입력 파일명을 단순화해 복사함
- ☑ 단순 이름으로 아티팩트 가져오기를 재시도함
- ☑ 엑셀 파일 구조와 데이터 시트 확인

```
cell_in[1], line 5
```

```
1 from artifact_tool import Blob, SpreadsheetFile
2 path="/mnt/data/260625_2000_2100_동해SS F3_전력품질_데이터.xlsx"
----> 3 wb = SpreadsheetFile.import_xlsx(Blob.load(path))
4 print(wb.inspect({"kind":"sheet","include":"id,name"}).ndjson)
```

```
File /tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/generated/interface/models.py:34:
34597 'Could not get source, probably due dynamically evaluated source code.'
```

```
File /tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/remote.py:913, in RemoteClas
```

```
File /tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/remote.py:983, in RemoteClas
979 'Could not get source, probably due dynamically evaluated source code.'
```

```
File /tmp/tmp.L2TH2Y5coc/artifact_tool_v2-2.8.22/artifact_tool/rpc/client.py:150, in JsonRpcCli
```

```
RemoteError: Format_InvalidStringWithValue, -394807792
```

참고: <https://chatgpt.com/share/6aaeb373-ec50-83ee-8423-cf2eaa840316>

VI. LLM 실습

- 로컬 환경의 필요성

× |← 코드 숨기기

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. 데이터 읽기
# -----
file_path = "input.xlsx" # 파일명 맞겨
sheet_name = "Sheet1" # 필요시 변경

df = pd.read_excel(file_path, sheet_name=sheet_name)

# 시간형 변환
df["시간"] = pd.to_datetime(df["시간"])

# 고조파 컬럼
low_harm_cols = [f"제{i}고조파" for i in range(1, 13)]
high_harm_cols = [f"제{i}고조파" for i in range(14, 50)]
all_harm_cols = low_harm_cols + high_harm_cols

# -----
# 2. 파생 변수 생성
# -----
df["저차고조파합_2_13"] = df[low_harm_cols].sum(axis=1)
df["고차고조파합_14_50"] = df[high_harm_cols].sum(axis=1)

# 분모 0 방지
df["저차_고차_비율"] = df["저차고조파합_2_13"] / df["고차고조파합_14_50"]
```

콘솔

실행 시작됨

Initializing environment

Installing packages

Running code

<exec>:1: DeprecationWarning:

Pyarrow will become a required dependency of pandas in the next major release of pandas (pandas 3.0),

(to allow more performant data types, such as the Arrow string type, and better interoperability with other libraries)

but was not found to be installed on your system.

If this would cause problems for you,

please provide us feedback at <https://github.com/pandas-dev/pandas/issues/54466>

FileNotFoundError: [Errno 44] No such file or directory: 'input.xlsx'

line 11, in <module>

File "/lib/python3.12/site-packages/pandas/io/excel/_base.py", line 495, in read_excel

io = ExcelFile(

^^^^^^^^^^^^

File "/lib/python3.12/site-packages/pandas/io/excel/_base.py", line 1550, in __init__

self._xltn = xltn if xltn is not None else None

VI. LLM 실습

- Codex 실행

The screenshot displays the Cursor IDE interface. On the left, a sidebar contains navigation options: 'New Chat', 'Search', 'Automations', 'Customize', 'Projects' (with a '+' icon), 'New Project' (with a circular arrow icon), 'Repositories' (with a list icon), and a file explorer showing 'cursor_test' and 'Harmonic data analysis' (with a '1m' indicator). The main workspace area is divided into two panes. The top pane shows a list of search results for '260625_2000_2100_동해SS F3_전력품질_데이터.xlsx', including 'Thought briefly' and 'Running Find the Excel file in common folders'. The bottom pane shows the terminal output of a PowerShell command: `Select-Object -ExpandProperty FullName; Get-ChildItem -Path "C:\Users\user\Documents\cursor_test", "C:\Users\user\Desktop", "C:\Users\user\Downloads" -Filter "*.xlsx" -Recurse -ErrorAction SilentlyContinue | Select-Object -ExpandProperty FullName`. The terminal output lists several file paths, including `C:\Users\user\Desktop\3_2.Research\3_0.Research_2023\3.??????.h\????\3.????\6530000\6530000_????U????g??_11_46_02_P_?i?????æü?.xlsx` and `C:\Users\user\Desktop\3_2.Research\3_0.Research_2023\3.??????.h\????\3.????\6530000\6530000_????U????g??_11_47_01_P_?????.xlsx`. On the right side of the IDE, a vertical toolbar contains icons for 'On cursor_test', 'Changes', 'Browser', 'Terminal', and 'Files'.

VI. LLM 실습

- Codex 결과 활용 챗GPT의 결과 보고서 작성

260625 _2000_2100_동해SS F3_전력품질_데이터.xlsx의 Sheet1 60행에 대해 첨부하신 코드를 실행했습니다.
robust z-score 종합점수가 가장 높은 시점은 20:41입니다.

커서

분석결과

- 장애 발생 전(20:40경) 전차선 전압 27.00[kV], 전차선 전류 67.1[A]
- 장애 발생 시(20:41경) 전차선 전압 97.36[kV], 전차선 전류 10,903.7[A]
- 절연파괴로 인한 저차 고조파(2~13차) 발생 확인
 - 평상시: 49차, 47차 고조파 전압

○ 사고시: 2~13차 고조파 전압 (2차 13.14, 3차 11.05가 가장 큼)

구분	시간	최소전압(V)	최대전압(V)	전류(A)	THD(%)	저차합(2~13)
사고 직전	20:40:28	26,883.91	26,999.25	67.1	6.31	3.63
사고 시점	20:41:28	218.49	97,360.30	10,903.7	1,495.92	70.88

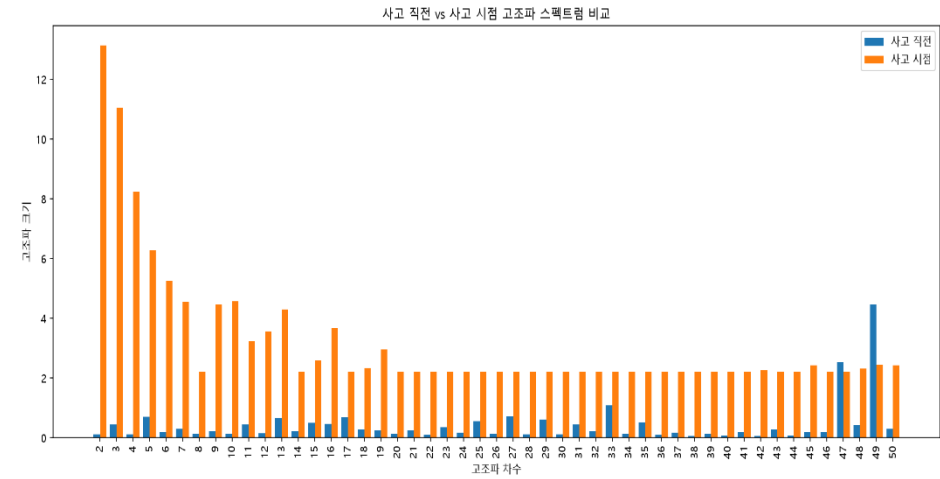
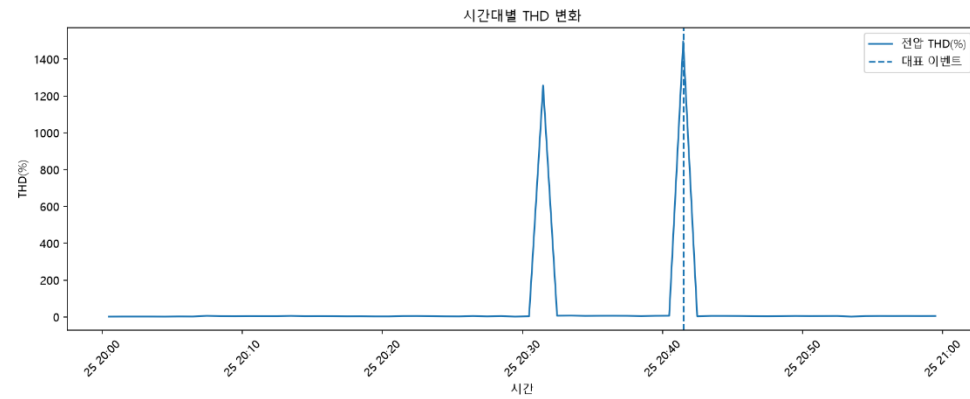
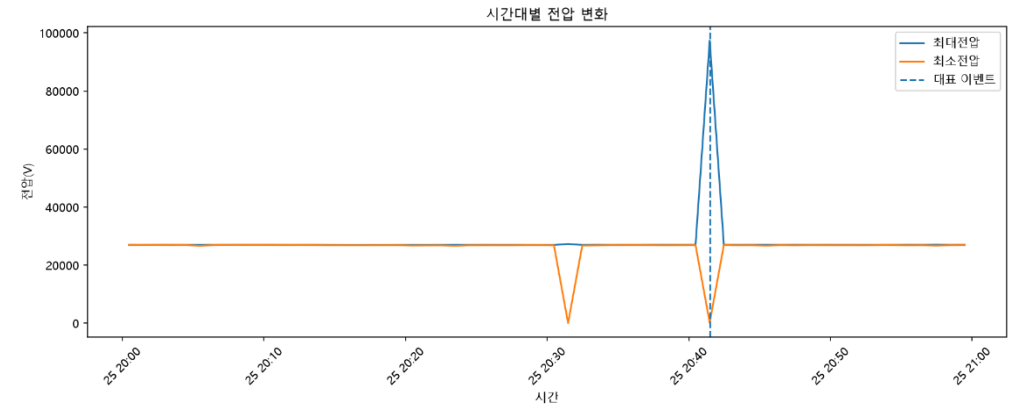
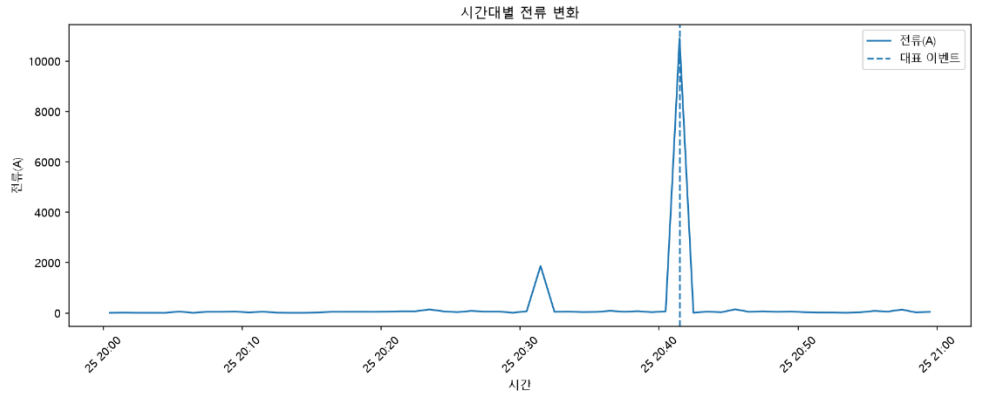
변화율: 전류 +16,145%, 최대전압 +261%, 최소전압 -99%, THD +23,607%, 저차고조파합 +1,853%.

20:41 값은 최소전압 붕괴·THD 1,496

📄 📌 📁 🔄 Just now

VI. LLM 실습

- 결과 보고서



VI. LLM 실습

- 결과 보고서

파일: 260625_2000_2100_동해SS F3_전력품질_데이터.xlsx
시트: Sheet1
행 수: 60

[이벤트 후보 상위 10개]

	시간	전류(A)	최대전압(V).1	최소전압(V).1	전압 THD(%)	저차고조파합_2_13	이벤트점수
2026-06-25	20:41:28.520	10903.695000	97360.300	218.494250	1495.92	70.88	6617.515754
2026-06-25	20:31:28.518	1869.529000	27313.445	23.367885	1256.61	125.80	2044.477826
2026-06-25	20:32:28.515	50.342990	26958.715	26772.615000	6.43	12.11	16.070934
2026-06-25	20:57:28.534	137.835050	27023.880	26746.575000	4.85	4.27	10.530792
2026-06-25	20:45:28.521	147.633050	26991.525	26732.820000	4.88	3.64	8.512597
2026-06-25	20:23:28.510	143.682550	26974.195	26682.810000	3.99	3.54	8.437148
2026-06-25	20:05:28.497	58.195945	26976.600	26697.450000	2.65	5.09	7.238274
2026-06-25	20:33:28.517	57.504465	26996.335	26821.615000	7.15	3.57	5.077194
2026-06-25	20:55:28.525	84.678260	27001.760	26832.065000	4.87	4.04	4.740693
2026-06-25	20:20:28.506	58.628225	26947.065	26778.690000	2.89	4.30	3.904581

[대표 이벤트]

직전 시점: 2026-06-25 20:40:28.521000

사고 시점: 2026-06-25 20:41:28.520000

[사고 직전 vs 사고 시점 비교]

구분	시간	최소전압(V).1	최대전압(V).1	전류(A)	전압 THD(%)	저차고조파합_2_13	고차고조파합_14_50	저차_고차_비율
사고 직전	2026-06-25 20:40:28.521	26883.91000	26999.25	67.120225	6.31	3.63	17.40	0.208621
사고 시점	2026-06-25 20:41:28.520	218.49425	97360.30	10903.695000	1495.92	70.88	85.32	0.830755

[변화율 요약]

전류 변화율(%): 16145.02

최대전압 변화율(%): 260.60

VI. LLM 실습

- 또 다른 분석

열별 승하차인원(26.01.01.-06.31) .xlsx
스프레드시트

이어서, 첨부된 엑셀 파일에 대해 분석을 하려고 해. 너가 할 수 있으면 여기서 하고, 아니면 로컬 vs코드에서 하도록 코드를 알려줘

🔍 스프레드시트 기술 문서와 API 빠른 시작 문서 검토

🔍 계약철도 승하차 데이터 구조 및 현황 분석 ▼

</> Python

```
import json, math, statistics, collections
res=wb.inspect({"kind":"table","range":f'"{sheet_name}"!A9:AE113',"include":"values","ta
data=json.loads(res.ndjson)["values"]
len(data), len(data[0]), data[0][:10], data[1][:10], data[2][:10]
```

(105,
31,
열별 승하차인원

VI. LLM 실습

- 챗GPT 분석

네, 이 파일은 여기서 충분히 분석할 수 있습니다. 데이터 규모도 작아서 로컬 VS Code로 옮길 필요가 없습니다. 제가 파일 구조를 먼저 확인해보니, 2026년 1~6월의 월별 열차 승하차 인원 자료이고, 실제 데이터 행은 102개입니다. 월별 열차번호 단위 자료에 역별 승차/하차가 가로로 배치된 형태입니다.

현재 파일에는 6개월, 17개 열차번호, 4개 열차종(새마을호·무궁화호·누리로·ITX-마음), 8개 역(철암·동백산·도계·신기·동해·묵호·정동진·강릉)이 포함되어 있습니다. 노선 구분은 1명동영주강릉과 중앙동대구-강릉 두 가지가 있습니다.

먼저 전체적인 결과를 간단히 보면, 1~6월 전체 승차인원은 75,150명, 하차인원은 75,372명, 승·하차 합계는 150,522명입니다.

역	승차	하차	승하차 합계	전체 비중
철암	27,486	28,720	56,206	37.3%
동해	16,705	15,818	32,523	21.6%
강릉	13,953	17,145	31,098	20.7%
묵호	5,661	3,954	9,615	6.4%
도계	4,807	...	9,437	6.3%

이 형태로 바꾸면 이후 분석이 굉장히 쉬워집니다.

제가 여기서 계속 진행한다면 다음 순서가 가장 좋습니다.

① long-format 데이터 정리 → ② 월별/역별 분석 → ③ 열차별 분석 → ④ 그래프와 히트맵 → ⑤ 결과 해석 및 보고서 문장 작성

파일 자체가 충분히 작기 때문에 제가 여기서 끝까지 분석할 수 있습니다. 다음 단계에서는 우선 ① 데이터를 분석용 long format으로 바꾸고, 역별·월별 상세 결과를 실제 수치로 뽑아보겠습니다.

📄 ⬆️ ↺️ ⋮

VI. LLM 실습

결과 보고서

월별로 보면 패턴도 꽤 뚜렷합니다.

월	승차	하차	승하차 합계	전월 대비	📄
1월	18,230	17,459	35,689	-	
2월	14,109	14,472	28,581	-19.9%	
3월	9,346	9,394	18,740	-34.4%	
4월	10,589	10,542	21,131	+12.8%	
5월	13,637	13,804	27,441	+29.9%	
6월	9,239	9,701	18,940	-31.0%	

즉, 1월이 가장 높고 → 3월까지 크게 감소 ↓ ~5월 회복 → 6월 다시 감소하는 형태입니다. 다만 이것을 바로 계절적 수요라고 해석하면 안 됩니다. 이 자료에는 월별 실제 운행 횟수나 운행일수가 없기 때문에, 승객수 변화가 수요 변화인지 열차 운행횟수 변화인지 분리할 수 없습니다.

열차종별을 보면 다음과 같습니다.

네. 여기서 직접 분석해서 분석용 long-format 데이터 + 요약표 + 대시보드까지 포함한 엑셀 파일로 만들었습니다.

핵심 결과는 다음과 같습니다.

- 2026년 1~6월 총 승하차: **150,522명**
 - 승차 75,150명
 - 하차 75,372명
- 역별로는 **철암 56,206명(37.3%)**, 동해 32,523명(21.6%), 강릉 31,098명(20.7%) 순이며, 상위 3개 역이 전체의 **79.6%**를 차지합니다.
- 월별 총 승하차는 **1월 35,689명 → 3월 18,740명으로 감소 → 5월 27,441명으로 회복 → 6월 18,940명으로 재감소**했습니다.
- 가장 변동이 큰 주요 역은 철암입니다. **1월 18,082명 → 3월 4,438명**으로 약 75% 감소한 뒤 5월 9,362명까지 회복했습니다.
- 동해는 4,787~6,717명 수준으로 상대적으로 안정적이고, 강릉은 **2월 6,314명 → 6월 4,008명**으로 감소하는 흐름입니다.
- 열차종별로는 **새마을호 87,931명(58.4%)**, 무궁화호 46,637명(31.0%), 누리로 12,076명(8.0%), ITX-마음 3,878명(2.6%)입니다.
- 승하차 불균형도 특징적입니다. 강릉은 하차가 승차보다 **3,192명 많고**, 묵호는 승차가 하차보다 **1,707명 많습니다**.

분석파일에는 LongData, 월별요약, 역별요 ↓ 열차종요약, 열차번호요약, 노선요약, 역월별, 대시보드 시트를 만들었습니다. 특히 원자료에서 동일 역이 두 노선 블록에 반복된 경

Industrial Data Science Lab

Contact:

won.sang.l@kangwon.ac.kr

<https://sites.google.com/view/idslab>